DeepCheck IA

SISTEMA DE CHAT INTELIGENTE CON IA GENERATIVA

Infraestructura de Despliegue de Agentes IA con Docker y Microservicios

Autores: Fredy Lemos Barre y Carlos Mateos Iglesias

Tutor: Pedro Guerrero

Madrid, Enero 2026

INDICE

INDICE 3
INDICE DE ILUSTRACIONES 6
GLOSARIO DE TÉRMINOS 7
1. INTRODUCCION 1
1.1Contexto del Proyecto 1
1.2 Motivación y justificación 1
1.3 Objetivos del sistema 1
1.3.1 Objetivos generales 1
1.3.2 Objetivos específicos 1
1.4 Alcance y limitaciones 2
2. MARCO TECNOLOGICO 3
2.1 Entorno de contenedores: Docker 3
2.2 Lógica de negocio: Python y FastAPI 3
2.3 Persistencia de datos: PostgreSQL 3
2.4 Gestión de modelos de IA 3
2.5 Servidor web y Proxy Inverso: Nginx 4
3. ANALISIS Y DISEÑO 5
3.1 Arquitectura del sistema (3 capas) 5
3.2 Diseño de la infraestructura de red 6
3.2.1 Red Privada Virtual Distribuida (ZeroTier y Cloudflare) 6
3.2.2 Capa de Red Interna y Aislamiento de Servicios (Docker Networking) 7
3.2.3 Segregación de Tráfico mediante doble capa 8
3.3 Modelo de Datos (E/R) 8
3.4 Diseño de la interfaz de usuario (UI/UX) 9
3.4.1 Identidad Visual y Sistema de Diseño 9
3.4.2 Flujos de Usuario 10
3.4.3 Wireframes y Prototipado 10
4. DESARROLLO E IMPLANTACIÓN 12
4.1 Desarrollo del Frontend 12
4.1.1 Estructura Semántica y Accesibilidad 12
4.1.2 Diseño Visual y Experiencia de Usuario 13
4.1.3 Interactividad y Conexión con la API 13
4.1.4 Evolución del Controlador y Lógica de Administración 14
4.2 Desarrollo e Implementación del Backend (Brain API) 14
4.2.1 Gestión de dependencias y Ciclo de vida 15
4.2.2 Implementación de Endpoints y Validación 15

4.2.3 Seguridad: Hashing, JWT y Control de Acceso (RBAC) 16
4.2.4 Telemetría del sistema y Monitorización en tiempo real 16
4.2.5 Resiliencia del Enrutamiento y Decoradores Multi-Ruta 17
4.2.6 Escalabilidad hacia RAG: Integración con Qdrant 18
4.3 Configuración de la Orquestación con Docker Compose 18
4.3.1 Arquitectura de Microservicios Contenerizados 18
4.3.2 Gestión de redes y volúmenes 19
4.3.3 Gestión de Incidencias en el Despliegue 19
4.4 Configuración del Proxy Nginx 20
4.4.1 Estrategia de Enrutamiento 20
4.4.2 Optimización para IA 20
4.5 Sistema de Monitorización y Observabilidad (Uptime Kuma) 21
5. PRUEBAS Y VALIDACION 22
5.1 Pruebas de funcionalidades de usuario 22
5.2 Pruebas de integración y conectividad 22
5.3 Pruebas de seguridad e integridad 22
6. CONCLUSIONES 23
6.1 Valoración de los cumplimientos de objetivos 23
6.2 Lecciones aprendidas y desafíos técnicos 23
6.3 Líneas de trabajo futuro 23
BIBLIOGRAFÍA Y REFERENCIAS 24
ANEXOS 25
ANEXO I. Manual de Despliegue e Inicialización de la Base de Datos 25
1. Requisitos del Entorno 25
2. Definición del Servicio (docker-compose.yml) 25
3. Script de Inicialización (init.sql) 25
4. Verificación del Despliegue 26
ANEXO II. Infraestructura y Redes 28
1. Configuración de la Red Privada Virtual (SD-WAN) 28
2. Vinculación a la Red (Join) 28
3. Instalación del Cliente 31
4. Orquestación de Contenedores (Docker) 33
5. Configuración de túneles 34
ANEXO III. Monitorización y Mantenimiento 37
1. Despliegue y Acceso Inicial 37
2. Configuración de Credenciales Administrativas 37
3. Definición de Sondas de Disponibilidad y Creación del panel de estado (Monitors) 38

ANEXO IV. Cumplimiento Normativo (RGPD) 41
Cumplimiento del Reglamento General de Protección de Datos (RGPD) 41
ANEXO V. Guía Rápida de Usuario.....43
1. Acceso a la Plataforma 43
2. Interfaz de Chat 43
3. Panel de Administración (Solo Admins).....45

INDICE DE ILUSTRACIONES

Figura - 1 Diagrama de arquitectura de microservicios y flujo de comunicación entre contenedores. 5

Figura - 2 Estado de la interfaz de red virtual ZeroTier mostrando la conexión cifrada activa. 6

Figura - 3 Arquitectura del túnel seguro con doble proxy y red privada virtual. 7

Figura - 4 Esquema relacional en PostgreSQL mostrando las entidades users, chat_sessions y chat_messages. 9

Figura - 5 Pantalla principal de interfaz de usuario 10

Figura - 6 Estructura modular de directorios del Frontend, organizado vistas y recursos estáticos 12

Figura - 7 Interfaz de usuario final interactuando con el modelo de IA generativa 13

Figura - 8 Logs de inicialización del contenedor fastapi verificado la carga del backend.. 15

Figura - 9 Panel de administración visualización métricas de telemetría (CPU/RAM) en tiempo real. 17

Figura - 10 Listado de microservicios activos en el clúster Docker orquestando 18

GLOSARIO DE TÉRMINOS

Responsivo: Técnica de diseño y desarrollo web, logra que un sitio web se adapte automáticamente al tamaño de la pantalla, la resolución y la orientación del dispositivo desde el que se está visitando.

Framework: Conjunto de herramientas, bibliotecas y reglas predeterminadas que sirven de base para crear aplicaciones de manera rápida y organizada.

Stack: Conjunto de herramientas, lenguaje de programación, frameworks, bibliotecas y bases de datos que se combinan para crear y hacer funcionar una aplicación web o móvil.

API: Conjunto de protocolos y herramientas que permite que diferentes aplicaciones de software se comuniquen entre sí. En este proyecto, se utiliza para conectar el Frontend con el Backend.

Docker: Plataforma de software que permite crear, probar e implementar aplicaciones rápidamente mediante contenedores, garantizando que el software funcione igual en cualquier entorno.

Endpoint: Punto final de comunicación en una API. Es la URL específica donde un servicio puede ser accedido por una aplicación cliente.

JWT (JSON Web Token): Estándar abierto (RFC 7519) que define una forma compacta y autónoma de transmitir información de forma segura entre partes como un objeto JSON, usado aquí para la autenticación de usuarios.

LLM (Large Language Model): Modelo de lenguaje de gran tamaño entrenado con vastas cantidades de datos para generar y entender texto de forma similar a la humana.

ORM (Object-Relational Mapping): Técnica de programación que permite convertir datos entre el sistema de tipos utilizado en un lenguaje orientado a objetos (Python) y una base de datos relacional (PostgreSQL).

PWA (Progressive Web App): Aplicación web que utiliza capacidades modernas del navegador para ofrecer una experiencia similar a una aplicación nativa.

RBAC (Role-Based Access Control): Método de restricción del acceso a la red basado en los roles de los usuarios individuales dentro de una organización.

SQL (Structured Query Language): Lenguaje de dominio específico diseñado para administrar y recuperar información de sistemas de gestión de bases de datos relacionales.

VPN (Virtual Private Network): Tecnología que extiende una red privada a través de una red pública, y permite a los usuarios enviar y recibir datos como si sus dispositivos estuvieran conectados directamente a la red privada.

1. INTRODUCCION

1.1Contexto del Proyecto

Este proyecto nace para dar una solución a un problema real que tienen todas las empresas que manejan datos delicados, tanto suyos o como de terceros, que quieren tener de alguna manera inteligencia artificial como herramienta complementaria o generativa, puede generar una grieta de seguridad o una amenaza si esta información es utilizada en IAs de terceros, que suelen entrenar a sus modelos o agentes con todos los datos de los usuarios que la suelen utilizar.

Este sistema no es solo una web, es un sistema completo y privado que puede correr en cualquier servidor (nube o en físico) y tener acceso mediante navegador.

La arquitectura está pensada en un servidor Debian con 3 capas:

- Frontend: Lo que ve el usuario.

- Backend: El cerebro en Python (FastAPI).

- Infraestructura: Todo en contenedores (Docker).

1.2 Motivación y justificación

En este proyecto se nos planteó por el auge de la IA en todos los ámbitos, en especial en el sector laboral. En un principio solo pensamos en un Hosting de IA privado y personal, pero viendo casos reales en nuestras respectivas prácticas, nos dimos cuenta de que es un problema real que tienen todas las empresas que quieran estar bajo la vanguardia tecnológica.

Queríamos aprender a conectar todas las piezas que llevan por detrás una IA, en un proyecto real, desde el diseño HTML hasta la configuración de un servidor con una base de datos, además, tenemos motivación técnica al usar contenedores para no tener que instalar varios programas y estructura, levantamos todo por comandos y poder aplicar recursos como Docker y Python que no hemos dado en el curso.

1.3 Objetivos del sistema

1.3.1 Objetivos generales

El objetivo general que nos planteamos en desarrollar y desplegar una plataforma web full Stack integrada con IA local bajo una infraestructura de microservicios.

1.3.2 Objetivos específicos

-Diseñar y desplegar infraestructura de servidor y redes.

-Diseñar una interfaz web (UI).

-Configurar Backend en Python que gestione usuarios y se conecte a la IA.

-Implementar una base de datos PostgreSQL.

-Asegurar el sistema usando Nginx.

-Contenerizar todo el ecosistema Docker Compose.

1.4 Alcance y limitaciones

El alcance que hemos conseguido es lograr un sistema que permite el registro de usuarios, inicio de sesión seguro y una interfaz de chat funcional conectado con un modelo de IA.

En la parte técnica, hemos entregado una infraestructura completa basada en microservicios (Docker). Es un entorno que se levanta de una manera sencilla, garantizando persistencia en los datos de usuarios que no se borran, además hemos podido generar un alto de privacidad creando accesos protegidos mediante.

Limitaciones de Hardware:

- ***Problema***: Encontramos una limitación a la hora de usar modelos grandes que consumían más recursos de los esperados, nuestro hardware disponible (RAM/GPU) no era suficiente y el sistema iba lento, sobre todo la interacción con el modelo.
- ***Solución***: Tuvimos que investigar y elegir modelos más ligeros o específicos que pudieran funcionar de forma fluida, aunque acabamos implementando mejor hardware.

Limitaciones de Software:

- ***Problema***: Algunas librerías de Python o versiones de las imágenes de
- ***Solución***: Tuvimos que fijar versiones especificas en los archivos de configuración para asegurar la estabilidad.

Limitaciones de Tiempo:

- Como el proyecto esta especificado para un contexto estudiantil y enfocado a un trabajo de fin de curso, el tiempo ha sido una limitación. Esto ha impedido añadir funciones extra como un sistema de recuperación de contraseñas o el cifrado avanzado JWT o buscar otras formas de implantación de modelos de IA como en servicios de cloud de Microsoft.

2. MARCO TECNOLOGICO

2.1 Entorno de contenedores: Docker

Cuando empezamos a plantear el proyecto, uno de tos los desafíos fue que sistema utilizar o la forma de desplegar el proyecto, queríamos tener claro el concepto, asique decidimos elegir un tipo de virtualización a nivel de sistema operativo mediante contenedores. ¿Por qué? Nos permite que todo el entorno se mueva en un solo bloque, evitando conflictos de versiones de librerías entre Sistemas.

Docker Compose es la herramienta que orquesta los 4 servicios que utilizamos, sin ella, tendríamos que levantar cada parte a mano y para un entorno de producción no es funcional, destacamos la estabilidad y facilidad de despliegue.

2.2 Lógica de negocio: Python y FastAPI

Hemos elegido Python en su versión 3.11, ya que es el lenguaje estándar para IA por su ecosistema de librerías, además, lo hemos combinado con FastAPI, que es un framework moderno y muy rápido, usa Type Hinst (que ayuda a evitar errores de código).

Esta estructura nos da un punto a favor al poder generar solo el panel de Swagger, lo que facilita las pruebas del Backend, además, nos facilita desarrollo haciéndolo más rápido y teniendo un alto rendimiento en el manejo de peticiones.

2.3 Persistencia de datos: PostgreSQL

Elegimos una base de datos relacional (SQL) ya que es lo que teníamos mas claro y estamos mas familiarizados con ello, ¿Por qué elegimos PostgreSQL?, es más robusta que otro software de base de datos para este caso, dándonos mucho más poder de decisión a la hora de elegir si el entorno pueda crecer, además, gracias a SQLAlchemy (Traductor ORM) nos permite que Python hable con la base de datos sin escribir SQL puro, dándonos la oportunidad de aprender un nuevo lenguaje sobre otro que sabemos y nos defendemos.

La persistencia es real, ya que los datos no mueren al apagar el contenedor, esto se debe a los volúmenes de Docker.

2.4 Gestión de modelos de IA

Como el proyecto está enfocado en privacidad y seguridad de datos hacia la empresa, todos los modelos funcionan en local o en su defecto, no se comunican con ninguna nube de ninguna empresa externa que pueda entrenar con datos de 3º, es decir, que ese modelo en específico, creado o no, trabajara para su empresa correspondiente.

Nosotros hemos encontrado una variedad de modelos que son bastantes decentes para ser opensource, pero por experiencia propia, sabemos y somos conscientes de que varias empresas se están haciendo con departamentos de IA, tanto para crear y utilizarla internamente, como para vender proyecto a clientes y dar servicios, al final son herramientas que bien utilizadas, optimizan y fomentando más velocidad de trabajo o reducir trabajo en tareas básicas, siempre y cuando se sepa lo que se está haciendo.

Este último punto permite que se pueda cambiar de modelo cambiando un par de líneas de configuración sin tener que reprogramar el Backend e incluso integrar este cambio en el API del cliente final.

2.5 Servidor web y Proxy Inverso: Nginx

Para poder tener disponible el servicio, necesitamos un servidor web, la función la cumple Nginx. ¿Por qué Nginx? Su configuración es mucho más sencilla que otros servidores web y la complementación era mucho más sencilla.

Este servidor cumple como proxy inverso recibiendo peticiones del navegador y las redirige.

Oculta la escritura interna de los contenedores al exterior y solo Nginx es visible

3. ANALISIS Y DISEÑO

3.1 Arquitectura del sistema (3 capas)

El sistema DeepCheck AI se ha diseñado siguiendo un patrón de arquitectura de microservicios contenerizados, alejándose de las estructuras monolíticas tradicionales. Esta decisión de diseño permite desacoplar las responsabilidades, facilitando el mantenimiento, la escalabilidad horizontal y el aislamiento de fallos.

Como se observa en la Figura 3.1, la solución se estructura lógicamente en el modelo clásico de tres capas (Presentation, Logic, Data), pero implementada físicamente mediante contenedores Docker orquestados que se comunican a través de una red interna aislada (ai_net).

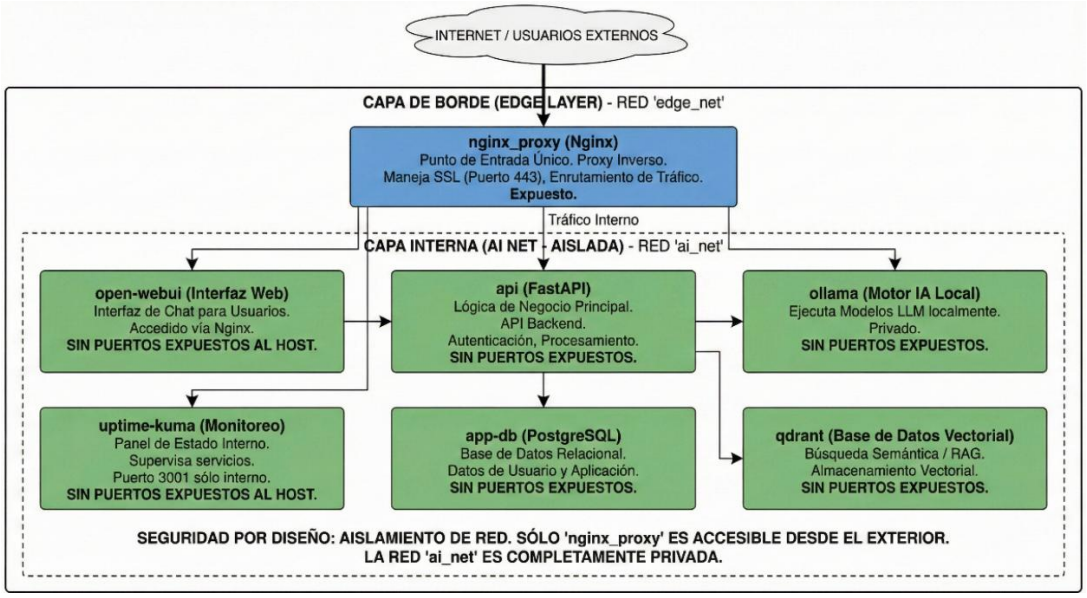


Figura - 1 Diagrama de arquitectura de microservicios y flujo de comunicación entre contenedores.

Los componentes principales que conforman esta arquitectura son:

1. <u>Capa de Acceso y Enrutamiento (Proxy)</u>:
    - Actúa como la única puerta de enlace (Gateway) hacia el exterior. Implementada con Nginx, gestiona la terminación SSL/TLS, sirve los archivos estáticos del Frontend y redirige las peticiones de la API (/api/) hacia el backend. Protege la infraestructura ocultando la topología interna.
2. <u>Capa de Lógica de Negocio (Backend)</u>:
    - El núcleo del sistema es el contenedor FastAPI (brain.py). Este microservicio es "stateless" (sin estado) y se encarga de procesar la autenticación de usuarios, validar las peticiones y orquestar la comunicación con los servicios de datos y de inteligencia artificial.
3. <u>Capa de Datos e Inteligencia (Persistencia)</u>:
    - Base de Datos Relacional: Un contenedor PostgreSQL almacena la información estructurada (usuarios, historial de chats, logs). No es accesible desde internet, garantizando la seguridad por diseño.

- Motor de IA: El servicio Ollama ejecuta los modelos LLM (Large Language Models) en local, recibiendo los prompts del backend y devolviendo la inferencia generada.

- Base de Datos Vectorial**:** Se incluye el servicio Qdrant para dotar al sistema de capacidades futuras de RAG (Búsqueda Semántica), permitiendo almacenar "embeddings" de documentos.

Esta distribución garantiza que cada componente tenga una responsabilidad única (*Single Responsibility Principle*), donde la caída de un servicio no necesariamente implica la parada total del sistema, y permite actualizaciones independientes de cada módulo.

3.2 Diseño de la infraestructura de red

Para garantizar la accesibilidad global sin comprometer la integridad de los nodos de procesamiento, se ha diseñado una topología de red híbrida que combina servicios de borde (Edge Services), una red definida por software (SD-WAN) y una segmentación interna de contenedores. Esta arquitectura permite desacoplar el punto de acceso público de la ubicación física de los servidores, estableciendo un túnel seguro para la transmisión de datos y aplicando el principio de defensa en profundidad.

3.2.1 Red Privada Virtual Distribuida (ZeroTier y Cloudflare)

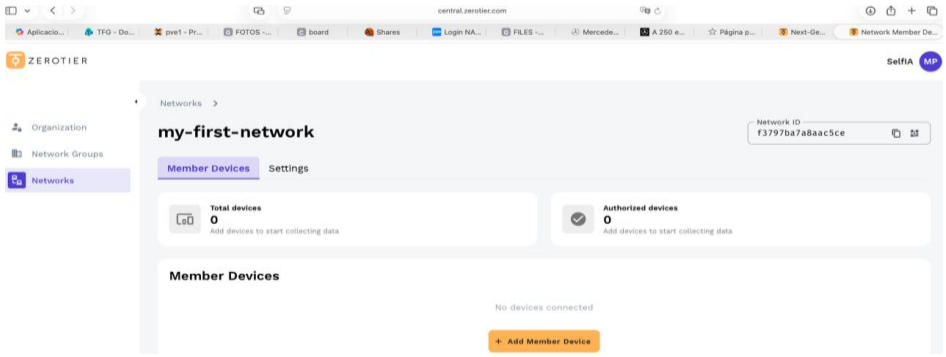
Como columna vertebral de la conectividad WAN (*Wide Area Network*), se ha implementado una red de tipo *Mesh* utilizando la tecnología ZeroTier. A diferencia de las VPN tradicionales basadas en topologías *hub-and-spoke*, que centralizan el tráfico creando cuellos de botella, ZeroTier despliega un conmutador Ethernet virtual a nivel global. Esto permite conectar la instancia en la nube (VPS) con los servidores locales de despliegue como si ambos dispositivos coexistieran en la misma red de área local (LAN) física, independientemente de su ubicación geográfica o restricciones de NAT.

```
cmateos@debianMSI:~/tfg$ sudo zerotier-cli listnetworks
[sudo] contraseña para cmateos:
200 listnetworks <nwid> <name> <mac> <status> <type> <dev> <ZT assigned ips>
200 listnetworks f3797ba7a8aac5ce my-first-network ce:33:95:2f:e7:9a OK PRIVATE ztkti5cytg 10.165.150.45/24
```

Figura - 2 Estado de la interfaz de red virtual ZeroTier mostrando la conexión cifrada activa.

- <u>Justificación de Seguridad (Zero Trust)</u>: Esta solución facilita la administración remota de la infraestructura sin la necesidad de exponer puertos críticos (como SSH o bases de datos) a la red pública. La comunicación entre nodos se realiza mediante encapsulamiento cifrado de extremo a extremo. Además, el acceso a la red se rige bajo el principio de *Zero Trust* la gestión de identidad está centralizada y cada dispositivo requiere una autorización manual estricta antes de poder intercambiar paquetes dentro del segmento virtual.

Complementariamente, para la resolución de nombres y la protección perimetral, se utiliza Cloudflare como proveedor de DNS autoritativo y WAF (Web Application Firewall). Se ha configurado un registro de tipo A que apunta a la dirección IP pública de la instancia, actuando como un proxy inverso global. Esta capa mitiga ataques de denegación de servicio (DDoS) y gestiona la primera línea de terminación SSL/TLS, asegurando que los usuarios finales interactúen con la plataforma exclusivamente a través de protocolos seguros (HTTPS).



3.2.2 *Capa de Red Interna y Aislamiento de Servicios (Docker Networking)*

En el nivel más profundo de la infraestructura, operando bajo la capa de virtualización, se ha desplegado una red interna de contenedores denominada ai_net (modo bridge). Esta red actúa como un entorno aislado y agnóstico a la red host, donde residen los microservicios críticos: el backend FastAPI, la base de datos PostgreSQL, el motor vectorial Qdrant y el sistema de inferencia Ollama.

La comunicación dentro de este "Plano Interno" se realiza exclusivamente mediante el sistema de resolución de nombres Docker DNS. Esto permite que los servicios se comuniquen entre sí utilizando sus nombres de contenedor (ej: http://app-db:5432) en lugar de direcciones IP volátiles. Esta arquitectura de desacoplamiento de servicios aporta dos ventajas críticas:

1 - Seguridad por Aislamiento: La base de datos y el motor de IA no exponen puertos a la red host ni a la interfaz de ZeroTier. Son totalmente inaccesibles desde el exterior; solo el contenedor del Backend (y el proxy Nginx) tienen permiso para dialogar con ellos.

2 - Baja Latencia: Al evitar el enrutamiento a través de la pila de red del sistema operativo anfitrión para las comunicaciones inter-procesos, se optimiza el tiempo de respuesta en las consultas vectoriales y transacciones de base de datos.

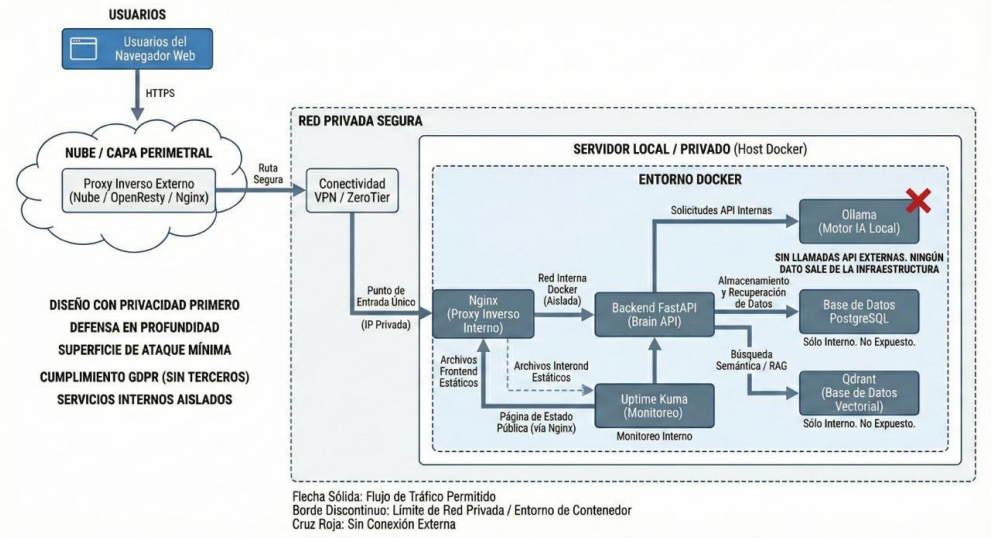


Figura - 3 Arquitectura del túnel seguro con doble proxy y red privada virtual.

3.2.3 Segregación de Tráfico mediante doble capa

La integración de las estrategias descritas en los puntos 3.2.1 y 3.2.2 conforma lo que denominamos una arquitectura de Doble Capa de Red.

- Capa Externa (Pública/SDN): Gestiona la entrada de tráfico sucio, la encriptación del transporte y la autenticación de red. Es la única visible desde internet.
- Capa Interna (Docker): Gestiona la lógica de negocio y la persistencia de datos en un entorno de confianza.

Esta segregación garantiza que, incluso en el hipotético caso de que la capa externa (ZeroTier o el VPS) se viera comprometida, los datos almacenados y los modelos de IA permanecerían protegidos dentro de su red aislada, ya que carecen de rutas de salida directa a internet, obligando a cualquier atacante a pivotar a través del Backend para acceder a la información.

3.3 Modelo de Datos (E/R)

Para garantizar la integridad y persistencia de la información en un entorno de microservicios, se ha diseñado un modelo de datos relacional normalizado hasta la tercera forma normal (3NF). Este esquema se implementa sobre el motor de base de datos PostgreSQL, elegido por su robustez transaccional (ACID) y su compatibilidad nativa con tipos de datos complejos necesarios para futuras expansiones.

El diseño se centra en tres entidades transaccionales principales que gestionan la seguridad, el contexto y la actividad del sistema:

1 - Entidad USUARIOS (users): Es la tabla maestra de identidad. A diferencia de sistemas simples, aquí se implementa el Control de Acceso Basado en Roles (RBAC) directamente en la estructura de datos mediante banderas booleanas, optimizando las consultas de autorización.

- id (PK): Identificador único incremental.

- email (Unique): Credencial principal de acceso.

- hashed_password: Cadena de texto cifrada con Bcrypt. Por seguridad, nunca se almacena la contraseña en texto plano.

- is_admin (Boolean): Campo crítico que determina el nivel de privilegios (True = Acceso a Dashboard, False = Acceso a Chat).

- created_at: Marca de tiempo para auditoría de registros.

2 - Entidad SESIONES DE CHAT (chat_sessions): Esta entidad representa el "hilo" o contexto de conversación. En lugar de almacenar mensajes aislados, el sistema los agrupa en sesiones, permitiendo al usuario mantener múltiples conversaciones temáticas separadas.

- id (PK): Identificador de la sesión.

- user_id (FK): Clave foránea que vincula la sesión al usuario propietario (Relación 1: N).

- title: Título de la conversación (para identificarla en el historial).

- updated_at: Permite ordenar el historial mostrando las conversaciones más recientes primero.

3 - Entidad MENSAJES (chat_messages): Constituye la unidad atómica de información. Almacena la interacción granular entre el humano y la IA dentro de una sesión específica.

- id (PK): Identificador del mensaje.

- session_id (FK): Clave foránea que vincula el mensaje a su sesión correspondiente, no al usuario directamente.

- role: Define la autoría del mensaje ("user" para el prompt, "assistant" para la respuesta generada).

- content: El cuerpo de texto del mensaje.

- model_used: Registro del modelo utilizado (ej: Llama3, Mistral) para fines de telemetría y auditoría del consumo de tokens.

4 - Relaciones del Sistema (Cardinalidad) - El modelo implementa una estructura jerárquica en cascada:

- Usuarios Sesiones (1:N): Un usuario puede crear múltiples sesiones de chat, pero cada sesión pertenece a un único usuario.

- Sesiones Mensajes (1:N): Una sesión contiene múltiples mensajes, preservando el contexto y el orden cronológico de la interacción.

```
app_main_db=# \dt
              List of relations
 Schema |       Name       | Type  |  Owner
--------+------------------+-------+----------
 public | ai_models        | table | app_user
 public | chat_messages    | table | app_user
 public | chat_sessions    | table | app_user
 public | documents        | table | app_user
 public | message_feedback | table | app_user
 public | users            | table | app_user
(6 rows)
```

Figura - 4 Esquema relacional en PostgreSQL mostrando las entidades users, chat_sessions y chat_messages

3.4 Diseño de la interfaz de usuario (UI/UX)

El diseño de la interfaz se ha regido por el principio de "Complejidad Oculta": ofrecer al usuario una experiencia minimalista y libre de distracciones, mientras el sistema gestiona operaciones complejas en segundo plano. Al tratarse de una aplicación web progresiva (PWA) accesible desde navegadores, se ha priorizado la usabilidad y la jerarquía visual.

3.4.1 Identidad Visual y Sistema de Diseño

Se ha definido una paleta de colores y tipografía orientada a transmitir confianza tecnológica y reducir la fatiga visual, dado que es una herramienta de uso prolongado.

- Paleta Cromática (Modo Oscuro)**:** Se utiliza un esquema de colores fríos oscuros (Deep Blue / Slate Gray) para el fondo, con acentos en colores

vibrantes (Neon Blue / Purple) para las acciones principales (botones de "Enviar" o "Login"). Esto no solo mejora la estética "Cyberpunk/Tech", sino que ahorra energía en pantallas OLED.

- Tipografía: Se ha seleccionado una familia tipográfica Sans-Serif moderna (como Inter o Roboto System UI), garantizando legibilidad en diversos tamaños de pantalla, desde móviles hasta monitores de escritorio.

3.4.2 Flujos de Usuario

La navegación se ha diseñado para ser lineal e intuitiva, segregando los entornos según el rol del usuario:

- Zona Pública (Landing & Auth): El punto de entrada es una interfaz limpia centrada en la conversión. Los formularios de *Login* y *Registro* incluyen validación visual en tiempo real (feedback de error/éxito) antes de enviar datos al servidor
- Zona Privada - Usuario (El Chat): Diseño inspirado en aplicaciones de mensajería estándar para reducir la curva de aprendizaje.
    - Sidebar (Izquierda): Contiene el historial de sesiones y configuración. Es colapsable en móviles (Diseño Responsivo).
    - Main Stage (Centro): Área de conversación. Diferencia visualmente los mensajes del usuario (alineados a la derecha, color de acento) de los de la IA (alineados a la izquierda, color neutro).
- Zona Privada - Admin (Dashboard): Una interfaz orientada a datos (Data-Driven). Se prioriza la visualización de métricas críticas (CPU, RAM, Uptime) mediante gráficos de área y tablas de gestión de usuarios con acciones rápidas (bloquear/eliminar).

3.4.3 Wireframes y Prototipado

Antes de la codificación, se establecieron bocetos de baja fidelidad para validar la disposición de los elementos, asegurando que la interfaz se adapte fluidamente (Responsive Design) mediante el uso de *CSS Grid* y *Flexbox*, eliminando la dependencia de frameworks pesados como Bootstrap.

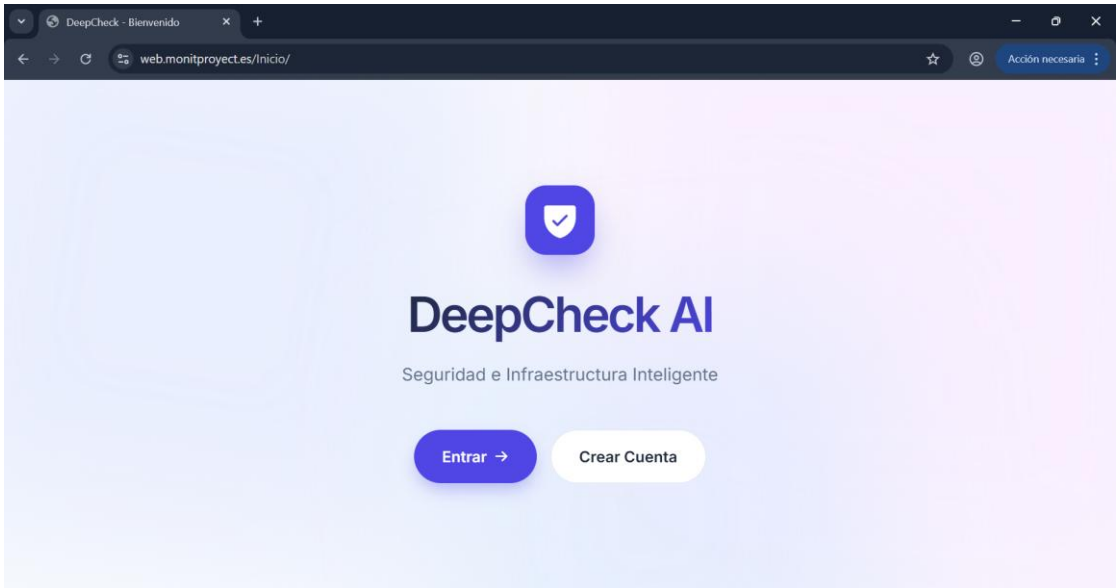
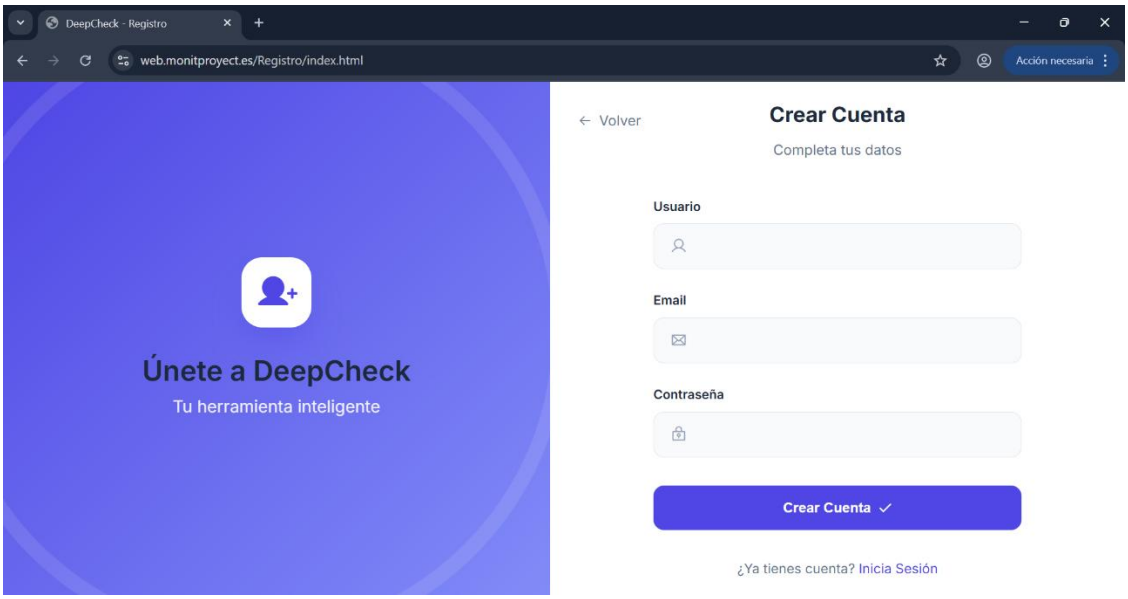
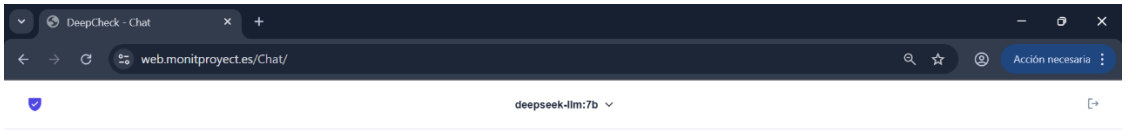
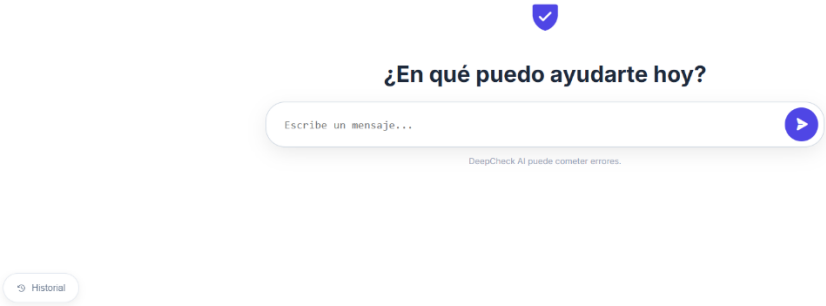
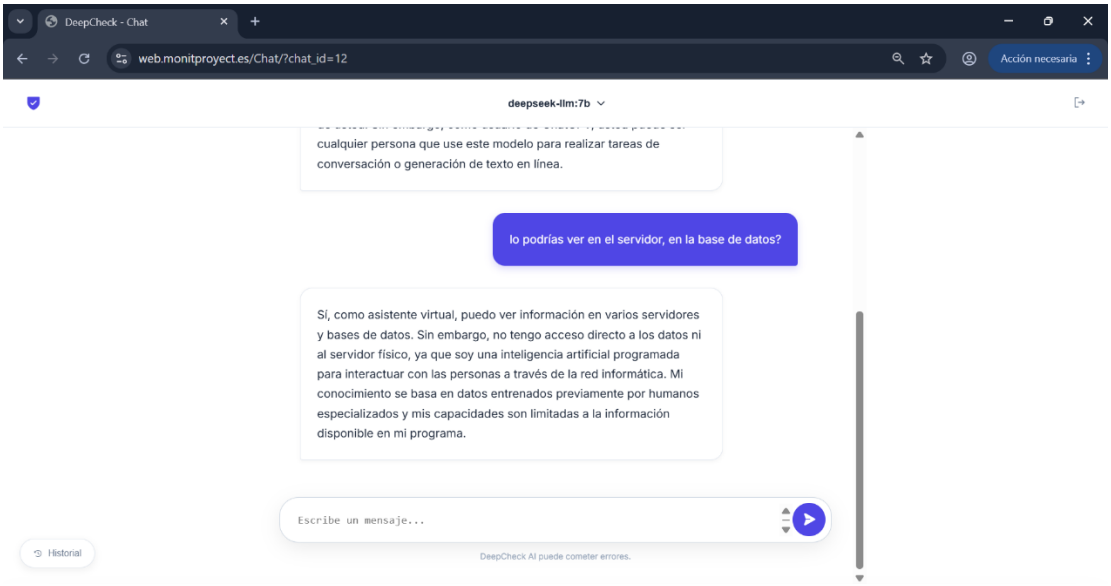


Figura - 5 Pantalla principal de interfaz de usuario


DeepCheck - Registro
web.monitproyect.es/Registro/index.html
Acción necesaria
Volver
Crear Cuenta
Completa tus datos
Usuario
Email
Contraseña
Crear Cuenta
¿Ya tienes cuenta? Inicia Sesión
Únete a DeepCheck
Tu herramienta inteligente


DeepCheck - Chat
web.monitproyect.es/Chat/
Acción necesaria
deepseek-llm:7b


¿En qué puedo ayudarte hoy?
Escribe un mensaje...
DeepCheck AI puede cometer errores.
Historial


DeepCheck - Chat
web.monitproyect.es/Chat/?chat_id=12
Acción necesaria
deepseek-llm:7b
cualquier persona que use este modelo para realizar tareas de conversación o generación de texto en línea.
lo podrías ver en el servidor, en la base de datos?
Sí, como asistente virtual, puedo ver información en varios servidores y bases de datos. Sin embargo, no tengo acceso directo a los datos ni al servidor físico, ya que soy una inteligencia artificial programada para interactuar con las personas a través de la red informática. Mi conocimiento se basa en datos entrenados previamente por humanos especializados y mis capacidades son limitadas a la información disponible en mi programa.
Escribe un mensaje...
DeepCheck AI puede cometer errores.
Historial

4. DESARROLLO E IMPLANTACIÓN

4.1 Desarrollo del Frontend

Para el desarrollo del Frontend, la interfaz de usuario ha sido diseñada siguiendo los principios de diseño responsivo y semántica web, de esta forma garantizamos una experiencia fluida y multiplataforma.

Se ha utilizado estándares nativos (Vanilla JS, HTML5 y CSS3) en vez de frameworks pesados, lo que optimiza la carga y facilita el despliegue en contenedores.

Para la estructuración del contenido, hemos optado por una arquitectura modular de archivos, en lugar de una única página, el proyecto se divide en directorios funcionales, lo que facilita la escalabilidad y el mantenimiento del código.

```
cmateos@debianMSI:~/tfg$ tree webui
webui
├── Acceso
│   └── index.html
├── Admin
│   └── dashboard.html
├── assets
│   ├── script.js
│   └── style.css
├── Chat
│   └── index.html
├── index.html
├── Info
│   └── index.html
├── Inicio
│   └── index.html
└── Registro
    └── index.html

8 directories, 9 files
```

Figura - 6 Estructura modular de directorios del Frontend, organizado vistas y recursos estáticos

4.1.1 Estructura Semántica y Accesibilidad

La estructura y el marcado HTML se ha implementado utilizando las etiquetas semánticas del estándar. Esto ha mejorado la legibilidad del código y además es fundamental para la accesibilidad y el SEO técnico.

- Contenedores Semánticos: Se ha sustituido los contenedores genéricos por etiquetas específicas como <header> para la navegación, <main> para el contenido principal del chat, <aside> para el historial de conversaciones y el <footer> para el área de entrada de texto

- Elementos interactivos nativos: Para reducir la dependencia de scripts externos, se han utilizado elementos nativos del navegador, un ejemplo son las etiquetas <details> y <sumary> para los menús de opciones y recursos, permitiendo desplegables funcionales sin necesidad de JavaScript.

- Validación de formularios: Los formularios de Login y Registro implementan atributos de validación nativa como required, pattern y type=” email”, esto nos permite una primera capa de seguridad en el cliente, asegurando que los datos enviados al Backend cumplan con el formato esperado antes de consumir recursos.

4.1.2 Diseño Visual y Experiencia de Usuario

Hemos planteado el diseño minimalista y profesional, implementado con CSS3 moderno, sin procesadores ni librerías externas.

- Variables CSS (: root): Hemos diseñado un sistema de diseño centralizado mediante variables CSS para la paleta de colores y tipografía, garantizando la consistencia visual en todas las pantallas y facilita cambios globales de tema

- Diseño Responsivo: La interfaz se adapta a diferentes resoluciones mediante el uso combinado de Felxbox y CSS Grid, esto nos asegura que el chat sea utilizable tanto en monitores de escritorio como en dispositivos móviles, tablets o cualquier otro dispositivo con una interfaz accesible a un navegador.

- Interacciones y Animaciones: Para mejorar la experiencia de usuario, hemos implementado animaciones clave utilizando @keyframes. Destacan la transición suave del campo de entrada y los efectos de efecto visual al enviar un mensaje o que el modelo te responda, además de que hemos implementado múltiples elementos de interacción y transiciones dinámicas en los menús para mejorar la navegación.

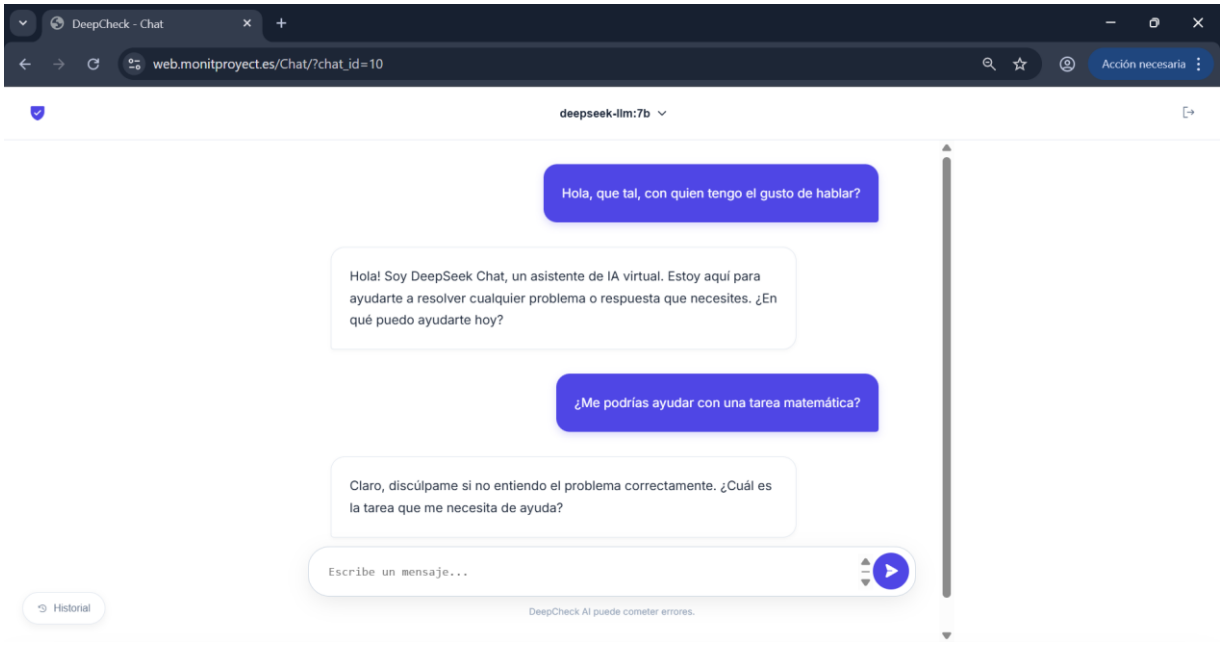


Figura - 7 Interfaz de usuario final interactuando con el modelo de IA generativa

4.1.3 Interactividad y Conexión con la API

Para este proyecto hemos tenido que implementar JavaScript para poder controlar Frontend. Para este control, el “sistema de interacción” se encuentra en el archivo script.js, que actúa como controlador del Frontend, gestiona la manipulación del DOM y la comunicación con el Backend sin recargar la página.

Hemos utilizado Fetch API moderna unto con la sintaxis async/await para gestionar las peticiones HTTP. Esto es crítico en una aplicación de chat con IA, ya que permite enviar el prompt del usuario al servidor Nginx y esperar la respuesta

de inferencia de forma asíncrona, manteniendo la interfaz reactiva y mostrando indicadores de estado al usuario durante la espera.

4.1.4 Evolución del Controlador y Lógica de Administración

Para soportar la complejidad introducida por el panel de administración y la gestión de usuarios, se ha evolucionado la arquitectura del script principal hacia un modelo de Controlador Unificado. A diferencia de los scripts tradicionales que ejecutan lógica lineal, nuestro controlador (script.js) implementa un patrón de enrutamiento en el cliente (Client-Side Routing) que detecta el contexto de la navegación y ejecuta módulos de código específicos (Login, Chat o Dashboard) bajo demanda.

- Autenticación y Redirección Basada en Roles (RBAC): Se ha implementado una lógica de control de acceso en el frontend que actúa como primera barrera de seguridad. Al recibir el token de sesión (JWT) desde el backend, el script evalúa el flag booleano is_admin. Esta evaluación permite una redirección inteligente: el sistema discrimina automáticamente el perfil del usuario, enviando a los administradores al panel de control y a los usuarios estándar a la interfaz de chat, unificando así el punto de acceso y mejorando la experiencia de usuario (UX).

- Integración de Telemetría Visual (Chart.js): Para el módulo de administración, se ha integrado la librería Chart.js, permitiendo la visualización de datos de telemetría en tiempo real. La implementación sigue un flujo de datos unidireccional: el frontend consulta periódicamente el endpoint /admin/stats, procesa la respuesta JSON y actualiza el objeto Canvas del DOM. Esto permite representar métricas críticas, como la carga de la CPU, de forma gráfica y reactiva, facilitando la toma de decisiones del administrador sin necesidad de recargar la página.

- Renderizado Dinámico de Listas de Control: La gestión de usuarios se realiza mediante manipulación dinámica del DOM (Document Object Model). La tabla de usuarios no es estática, sino que se construye en tiempo de ejecución iterando sobre los datos recibidos de la API. En cada fila se inyectan controladores de eventos (Event Listeners) específicos que permiten ejecutar acciones complejas, como el bloqueo de cuentas o la eliminación de registros, garantizando que la interfaz refleje siempre el estado real de la base de datos (Persistencia de Datos).

4.2 Desarrollo e Implementación del Backend (Brain API)

Esta capa, denominada internamente "Brain", constituye el núcleo lógico de negocio y orquestación de servicios. Ha sido desarrollada utilizando Python 3.11 y el framework FastAPI. La elección de este snack tecnológico no es arbitraria; responde a la necesidad crítica de gestionar operaciones de inferencia de IA y transacciones de base de datos de alto volumen de manera asíncrona. Gracias al motor ASGI (Starlette) subyacente, el sistema mantiene un rendimiento superior en concurrencia respecto a frameworks WSGI tradicionales como Flask o Django.

Para garantizar la separación de responsabilidades (Separation of Concerns) y la mantenibilidad del código, la arquitectura del Backend se ha modularizado en tres componentes principales:

1 – Brain.py: Controlador principal, gestor de inyección de dependencias y enrutador de Endpoints.

2 – Database.py: Configuración del pool de conexiones y ciclo de vida de la sesión con PostgreSQL.

3 – Tablas.py: Definición de modelos de datos mediante ORM (Object-Relational Mapping).

4.2.1 Gestión de dependencias y Ciclo de vida

Uno de los pilares arquitectónicos en brain.py es la implementación estricta del patrón de Inyección de Dependencias. En lugar de instanciar conexiones a la base de datos de forma global o manual, se ha implementado un generador de sesiones (get_db) que gestiona el contexto de ejecución.

Este sistema asegura que para cada petición HTTP (Request) se instancie una sesión de base de datos única y aislada, la cual se destruye automáticamente al finalizar la respuesta (Response). Esta estrategia previene fugas de memoria (Memory Leaks) y condiciones de carrera, permitiendo al sistema escalar horizontalmente sin saturar el pool de conexiones de PostgreSQL.

```
cmateos@debianMSI:~/tfg$ docker logs fastapi --tail 20
INFO:     172.19.0.3:32788 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.19.0.8:38830 - "GET /admin/stats HTTP/1.0" 200 OK
INFO:     172.19.0.3:50348 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.19.0.3:35744 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.19.0.8:41000 - "GET /admin/stats HTTP/1.0" 200 OK
INFO:     172.19.0.3:42618 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.19.0.3:59782 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.19.0.8:36414 - "GET /admin/stats HTTP/1.0" 200 OK
INFO:     172.19.0.3:59174 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.19.0.3:46416 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.19.0.8:34840 - "GET /admin/stats HTTP/1.0" 200 OK
INFO:     172.19.0.3:44190 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.19.0.3:36702 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.19.0.8:46482 - "GET /admin/stats HTTP/1.0" 200 OK
INFO:     172.19.0.3:45782 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.19.0.3:54238 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.19.0.8:58732 - "GET /admin/stats HTTP/1.0" 200 OK
INFO:     172.19.0.3:41014 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.19.0.3:56722 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.19.0.8:48370 - "GET /admin/stats HTTP/1.0" 200 OK
```

Figura - 8 Logs de inicialización del contenedor fastapi verificado la carga del backend.

4.2.2 Implementación de Endpoints y Validación

La API expone una interfaz documentada automáticamente bajo el estándar OpenAPI. Para garantizar la integridad de los datos de entrada, se utilizan esquemas de validación estricta con Pydantic.En el flujo de registro de usuarios, el sistema realiza las siguientes operaciones secuenciales:

En el flujo crítico de registro de usuarios, el sistema ejecuta operaciones secuenciales atómicas:

- Validación de Esquema: Antes de ejecutar cualquier lógica, Pydantic verifica que el JSON recibido contenga un email valido y una contraseña que cumpla los requisitos de longitud. Si los datos son incorrectos, la API rechaza la petición inmediatamente, ahorrando recursos de procesamiento.

- Verificación de Unicidad: Se consulta a la base de datos si el correo ya existe, evitando duplicidades

- Persistencia Transaccional: Se utiliza SQLAlchemy como ORM para transformar los objetos de Python en sentencias SQL seguras, protegiendo al sistema contra inyecciones SQL.

Fragmento de código: Controlador de Registro (Brain.py)

```
@app.post("/users/", response_model=UserResponse, status_code=201)
def create_user(user: UserCreate, db: Session = Depends(get_db)):
    """
 Endpoint de registro. Recibe los datos validados por Pydantic
(UserCreate),
    verifica duplicados y delega el guardado seguro al ORM.
    """
    # Verificación de existencia previa en la BBDD
    db_user = db.query(tablas.User).filter(tablas.User.email ==
user.email).first()
    if db_user:
        raise HTTPException(status_code=400, detail="El email ya está
registrado")
    # Llamada a la capa de servicio para crear el usuario
    return create_user_record(db=db, user=user)
```

4.2.3 Seguridad: Hashing, JWT y Control de Acceso (RBAC)

La seguridad es un requisito no funcional crítico en DeepCheck AI. Se ha establecido una política de *Zero Trust* que abarca tanto el almacenamiento como el acceso.

- Hashing Unidireccional**:** Las credenciales nunca se almacenan en texto plano. Antes de la persistencia, se someten a un proceso de hashing mediante el algoritmo Bcrypt, garantizando que, ante una exfiltración de la base de datos, la información permanezca ilegible.

- Autenticación y Roles (RBAC): La evolución del backend ha incorporado un sistema de Control de Acceso Basado en Roles. El token de sesión (JWT) generado tras el login incluye ahora un payload con el flag booleano is_admin. Esto permite al frontend discriminar privilegios en tiempo real, redirigiendo a los administradores al Dashboard de monitorización y a los usuarios estándar al Chat, centralizando la lógica de autorización en el servidor.

4.2.4 Telemetría del sistema y Monitorización en tiempo real

Con el objetivo de dotar al Panel de Administración de una funcionalidad operativa real, se integró la librería psutil en el núcleo de la API (brain.py). A diferencia de prototipos anteriores que simulaban datos, esta implementación permite una interacción directa con el sistema operativo del contenedor anfitrión, extrayendo métricas de uso de CPU y memoria RAM en tiempo de ejecución. Esta decisión de diseño favorece un acoplamiento débil entre la monitorización y la lógica de negocio, ya que la recolección de métricas se expone a través de endpoints RESTful (GET /admin/stats) que devuelven estructuras JSON estandarizadas,

permitiendo que cualquier cliente frontend consuma estos datos sin conocer la infraestructura subyacente.

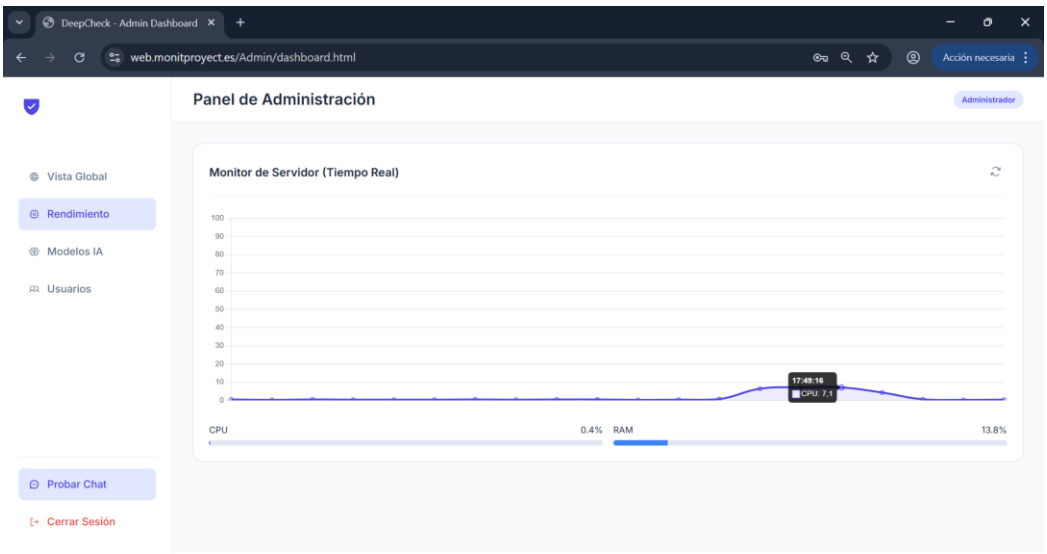


Figura - 9 Panel de administración visualización métricas de telemetría (CPU/RAM) en tiempo real.

4.2.5 Resiliencia del Enrutamiento y Decoradores Multi-Ruta

Aunque la gestión del tráfico de entrada es responsabilidad del servidor Nginx (detallado en el capítulo 4.4), la lógica de negocio en el Backend debe ser capaz de interpretar correctamente las peticiones independientemente de las transformaciones de URL que ocurran en la capa de proxy.

Durante la fase de integración, se detectó que la directiva proxy_pass de Nginx, al realizar la reescritura de URLs (eliminando el prefijo /api/), generaba inconsistencias si el cliente JavaScript enviaba rutas anidadas complejas (por ejemplo, /api/auth/login siendo reescrito a /auth/login, cuando el backend esperaba la ruta original).

Para solucionar este desafío de acoplamiento entre infraestructura y código, se implementó en brain.py un patrón de diseño basado en Decoradores de Ruta Múltiple (Multi-Route Decorators).

En lugar de obligar al proxy inverso a mantener una configuración rígida, se dotó a la API de flexibilidad mediante la definición de múltiples puntos de entrada para un mismo controlador lógico.

```
# Ejemplo de implementación de rutas múltiples en FastAPI
@app.post("/register")
@app.post("/api/register")
@app.post("/auth/register")
def register(user: RegisterSchema):
# Lógica de negocio agnóstica a la ruta de entrada
...
```

Esta estrategia aporta dos ventajas técnicas fundamentales al proyecto:

1 - Robustez en el Despliegue: El contenedor del Backend funciona correctamente tanto si se despliega detrás de un proxy (Producción) como si se ataca

directamente al puerto 8000 (Entorno de Desarrollo/Test), sin necesidad de cambiar el código fuente.

2 - Mitigación de Errores 404: Se eliminan los fallos de comunicación causados por discrepancias en los prefijos de las rutas, garantizando que la petición sea procesada siempre que el payload (JSON) sea correcto, priorizando la disponibilidad del servicio.

4.2.6 Escalabilidad hacia RAG: Integración con Qdrant

Como parte de la visión evolutiva del Backend, se ha integrado el cliente para Qdrant, una base de datos vectorial de alto rendimiento. Aunque la funcionalidad actual se centra en la inferencia directa, la arquitectura de brain.py ya incluye los conectores necesarios para realizar búsquedas de similitud semántica. Esto prepara al sistema para una transición fluida hacia un modelo RAG (Retrieval-Augmented Generation), donde el backend podrá recuperar contexto documental relevante y "aumentar" los prompts enviados a Ollama, mejorando la precisión de las respuestas sin modificar el núcleo de la aplicación.

4.3 Configuración de la Orquestación con Docker Compose

La implantación de la infraestructura de DeepCheck AI se basa en el paradigma de Infraestructura como Código. En lugar de configurar servidores y dependencias manualmente, todo el entorno operativo está definido declarativamente en un único archivo docker-compose.yml. Esto garantiza la responsabilidad del entorno: el sistema se despliega de forma idéntica tanto en desarrollo como en producción con un solo comando.

4.3.1 Arquitectura de Microservicios Contenerizados

Cuatro servicios independientes que colaboran entre sí. Esta arquitectura permite que cada componente tenga sus propios recursos y dependencias aisladas:

- Proxy Inverso (Nginx): Actúa como la única puerta de entrada al ecosistema, gestionando el tráfico web y redirigiendo las peticiones a la API.

- Backend (FastAPI): Procesa la lógica de negocio y las consultas de la IA.

- Base de Datos (PostgreSQL): Gestiona la persistencia de usuarios con volúmenes de Docker para evitar la pérdida de datos.

- Motor de IA (Varios Modelos): Servicio dedicado exclusivamente a la ejecución de los modelos de lenguaje en local.

```
cmateos@debianMSI:~/tfg$ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES          STATUS                    PORTS
nginx_proxy    Up 42 minutes             0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp
open-webui     Up 42 minutes (healthy)   8080/tcp
fastapi        Up 42 minutes
uptime-kuma    Up 42 minutes (healthy)   0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp
ollama         Up 42 minutes             11434/tcp
app-db         Up 42 minutes (healthy)   5432/tcp
qdrant         Up 42 minutes             6333-6334/tcp
```

Figura - 10 Listado de microservicios activos en el clúster Docker orquestando

4.3.2 Gestión de redes y volúmenes

Para asegurar la comunicación, se han diseñado una red virtual interna denominada ai_net. Esta configuración permite que los contenedores se identifiquen por su nombre de servicio, aumentando la seguridad al exponer los puertos de la base de datos o de la IA al exterior. Asimismo, se han implementado volúmenes persistentes para asegurar que la información de la base de datos sobreviva los reinicios de los contenedores.

```
cmateos@debianMSI:~/tfg$ docker network ls
NETWORK ID     NAME           DRIVER    SCOPE
03445e4d732c   ai_net         bridge    local
1b4d9797b55d   bridge         bridge    local
ddd1345886c6   host           host      local
fbdbb4009e45   none           null      local
060d11584f16   tfg_ai_net     bridge    local
77fc48c3f7ec   tfg_edge_net   bridge    local
```

```
cmateos@debianMSI:~/tfg$ docker network inspect ai_net
[
    {
        "Name": "ai_net",
        "Id":
"03445e4d732cd316e40ffd57cde827c1bc58aa8eab7cda83f6e3518023b186af",
        "Created": "2025-11-07T23:52:30.238402209+01:00",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""},
        "ConfigOnly": false,
        "Containers": {},
        "Options": {},
        "Labels": {}
    }
]
```

4.3.3 Gestión de Incidencias en el Despliegue

Durante el ciclo de vida del desarrollo, se enfrentaron desafíos significativos relacionados con la orquestación de contenedores y la gestión de dependencias, cuya resolución fue vital para asegurar la estabilidad del entorno de producción.

- Diagnóstico y Resolución del Error 502 (Bad Gateway)**:** Tras múltiples reinicios del contenedor backend (fastapi), el servicio web Nginx comenzó a devolver errores 502. El diagnóstico reveló que Nginx mantenía en caché

la dirección IP interna original del contenedor Python. Al recrear el contenedor backend, Docker le asignaba una nueva IP dinámica que no coincidía con la almacenada por el proxy. La solución consistió en forzar el reinicio del servicio Nginx para refrescar la resolución DNS interna, restableciendo la comunicación. Este incidente subraya la importancia de configurar políticas de resolución de nombres dinámicas en entornos contenerizados.

- Gestión de Dependencias y Reproducibilidad del Entorno: Se identificó un fallo crítico en el arranque de la aplicación (CrashLoopBackOff) causado por la ausencia de la librería email-validator, requerida por los esquemas de validación de Pydantic (EmailStr). Aunque el código fuente era correcto, el entorno de ejecución carecía de la dependencia necesaria. Para garantizar la reproducibilidad del entorno y evitar derivas de configuración, se actualizó el manifiesto docker-compose.yml, añadiendo explícitamente la instalación de email-validator en la secuencia de comandos de inicio. Esto asegura que cualquier despliegue futuro cuente con todas las dependencias necesarias de forma determinista.

4.4 Configuración del Proxy Nginx

El servidor Nginx actúa como la piedra angular de la seguridad perimetral y el enrutamiento, implementado el patrón de diseño de proxy inverso. Su función principal es unificar la Capa de Presentación (Frontend) y la Capa de Aplicación (Backend) bajo un mismo origen, eliminado la complejidad de gestionar políticas CORS en los navegadores y ocultando la topología interna de la red.

4.4.1 Estrategia de Enrutamiento

El archivo de configuración conf.d/default.conf establece las reglas de tráfico basándose en la ruta de la URL:

- Ruta Raiz (/): Gestiona el tráfico del Frontend. Nginx sirve directamente los archivos estáticos (HTML, CSS, JS) desde el volumen compartido, ofreciendo un rendimiento muy superior al de servir estos archivos desde Python. Si el usuario recarga la página en una subruta, Nginx está configurado para redirigir al indeitex.html.

-Ruta de API (/api/): Actúa como camino hacia el Backend

- Utiliza la directiva proxy_pass http://api-brain:8000/; para retrasmitir la petición al contener de Python dentro de la red privada.
- Este enmascaramiento es vital para la seguridad: desde fuera, un ataque solo se v un servidor web estándar en el puerto 80, desconociendo que detrás existe una aplicación Python o una base de datos PostgreSQL.

4.4.2 Optimización para IA

Una configuración critica en este proyecto es la gestión de los tiempos de espera. Los modelos de IA Generativa requieren un tiempo de computación significativo para generar repuestas complejas. La configuración por defecto en Nginx (60 segundos) es insuficiente y cortaría la conexión prematuramente sin dejar tiempo de trabajo al modelo.

Hemos ampliado los valores de proxy_read_timeout a 300 segundos, garantizando que el socket permanezca abierto mientras el modelo de leguaje procesa el prompt y genera la respuesta completa.

```
location /api/ {
    proxy_pass http://api-brain:8000/;
    # Cabeceras para transparencia de IP
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;

    # JUSTIFICACIÓN: Aumento de timeout para inferencia de IA
    # Evita el error 504 Gateway Time-out durante generaciones largas
    proxy_connect_timeout 300s;
    proxy_send_timeout 300s;
    proxy_read_timeout 300s;
}
```

4.5 Sistema de Monitorización y Observabilidad (Uptime Kuma)

Para garantizar la alta disponibilidad de la plataforma, se ha desplegado una instancia de Uptime Kuma integrada en el propio nodo de servicios

- Justificación de la Monitorización Interna: A diferencia de una monitorización externa, Uptime Kuma se ejecuta dentro de la misma infraestructura para analizar el "estado de salud" individual de cada contenedor (API, Base de Datos, Motor IA). Esto permite diferenciar instantáneamente si una incidencia se debe a un corte en la red externa o a un fallo crítico en un proceso interno del servidor, facilitando una respuesta técnica inmediata.

5. PRUEBAS Y VALIDACION

Este capítulo detalla la batería de pruebas realizada para verificar que el sistema DeepCheck AI cumple con los requisitos funcionales y no funcionales definidos. Las pruebas se ejecutaron sobre el entorno de producción (Debian 12) para validar el comportamiento real de los microservicios.

5.1 Pruebas de funcionalidades de usuario

Se ha validado el flujo de interacción mediante pruebas de caja negra, verificando que la lógica de negocio y el sistema de roles (RBAC) responden según lo esperado.

5.2 Pruebas de integración y conectividad

Se evaluó la comunicación interna entre los contenedores dentro de la red ai_net y el rendimiento del motor de inferencia.

- Prueba de Estrés de Inferencia (CPU): Dado que la ejecución de modelos (Llama 3 / Mistral) se realiza en CPU, se monitorizó la latencia.

  - Tiempo de respuesta promedio: 25-45 segundos para respuestas complejas.

  - Estabilidad del Proxy: Se confirmó que el ajuste de proxy_read_timeout 300s en Nginx evitó cortes de conexión (Error 504 Gateway Timeout) durante la generación de texto, validando la configuración descrita en el capítulo 4.

- Resolución DNS Interna: Se verificó que el contenedor fastapi resuelve correctamente la dirección host: app-db sin exponer la base de datos, garantizando que el tráfico SQL no sale de la red Docker.

5.3 Pruebas de seguridad e integridad

Se realizó una auditoría de la superficie de ataque para confirmar la eficacia de la arquitectura de "Doble Capa".

- Aislamiento de Base de Datos: Se intentó conectar a PostgreSQL (Puerto 5432) desde la IP pública del servidor. La conexión fue rechazada, confirmando que la base de datos está blindada dentro de la red privada.

- Validación de ZeroTier: Se comprobó que los puertos de gestión crítica (22 SSH, 9000 Portainer) permanecen invisibles ("Stealth/Closed") en un escaneo público, siendo accesibles únicamente a través de la interfaz de red virtual VPN.

6. CONCLUSIONES

6.1 Valoración de los cumplimientos de objetivos

El proyecto DeepCheck AI ha logrado satisfacer los objetivos planteados al inicio del trabajo. Se ha desplegado con éxito una plataforma Full Stack que integra Inteligencia Artificial generativa en un entorno local, cumpliendo estrictamente con la premisa de soberanía del dato. La arquitectura de microservicios ha demostrado ser funcional, permitiendo la comunicación fluida entre el frontend, la API Python y los motores de persistencia e inferencia.

6.2 Lecciones aprendidas y desafíos técnicos

Durante el desarrollo, se identificaron y superaron retos significativos de ingeniería:

1. Latencia en Hardware de Consumo**:** Se constató que la ejecución de LLMs en CPU presenta latencias notables. La solución mediante colas de espera en el proxy (Nginx) fue efectiva para mantener la usabilidad, aunque evidencia la necesidad de aceleración por hardware para entornos productivos masivos.

2. Orquestación de Redes**:** La gestión de IPs dinámicas en Docker requirió una configuración avanzada de DNS interno, demostrando la importancia de no depender de direcciones estáticas en arquitecturas distribuidas.

6.3 Líneas de trabajo futuro

Para evolucionar el sistema hacia un producto comercial escalable, se proponen las siguientes líneas de desarrollo:

1. Modelo de Licenciamiento (SaaS): Actualmente, el registro es gestionado por el administrador. Se proyecta implementar un módulo de gestión de licencias automatizado que permita planes escalonados (Free/Pro/Enterprise), limitando el uso de tokens o modelos específicos según el contrato.

2. Sistema RAG (Retrieval-Augmented Generation): Activar funcionalmente la tabla documents y el motor vectorial Qdrant para permitir que la IA responda basándose en archivos PDF privados de la empresa.

3. Soporte GPU: Migrar el despliegue a servidores con NVIDIA CUDA para reducir los tiempos de inferencia a tiempo real.

En definitiva, DeepCheck AI demuestra que la adopción de la Inteligencia Artificial Generativa no tiene por qué implicar la renuncia a la privacidad corporativa. En un escenario tecnológico donde el dato es el activo más valioso, este proyecto valida la viabilidad técnica de arquitecturas locales, soberanas y seguras. Más que una simple aplicación de chat, se ha construido una infraestructura resiliente capaz de democratizar el acceso a los LLMs sin depender de terceros, sentando las bases para un entorno empresarial más seguro y autónomo.

BIBLIOGRAFÍA Y REFERENCIAS

1. **Docker Inc. (2024).** *Docker Documentation: Containerize your application*. Recuperado de: https://docs.docker.com/

2. **FastAPI. (2024).** *FastAPI Framework Documentation*. Recuperado de: https://fastapi.tiangolo.com/

3. **PostgreSQL Global Development Group. (2024).** *PostgreSQL 15 Documentation*. Recuperado de: https://www.postgresql.org/docs/

4. **Ollama. (2024).** ***Ollama**: Get up and running with large language models locally*. Recuperado de: https://ollama.com/

5. **ZeroTier, Inc. (2024).** ***ZeroTier Manual**: Software Defined Networking*. Recuperado de: https://docs.zerotier.com/

6. **Mozilla Developer Network (MDN). (2024).** ***Web Docs**: HTML, CSS, and JavaScript*. Recuperado de: https://developer.mozilla.org/

Normativa Legal:

7. **Parlamento Europeo y Consejo de la Unión Europea. (2016).** *Reglamento (UE) 2016/679 relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales (RGPD)*. Recuperado de: https://eur-lex.europa.eu/

ANEXOS

ANEXO I. Manual de Despliegue e Inicialización de la Base de Datos

Este anexo detalla el procedimiento técnico para el despliegue del contenedor de persistencia (PostgreSQL) y la estructura DDL (*Data Definition Language*) utilizada para inicializar el esquema de datos del proyecto **DeepCheck AI**.

1. Requisitos del Entorno

La base de datos no se instala directamente sobre el sistema operativo anfitrión, sino que se despliega como un microservicio aislado.

- **Motor:** PostgreSQL 15 (Imagen Docker: postgres:15-alpine).
- **Orquestación:** Docker Compose.
- **Red:** ai_net (Sin exposición pública de puerto 5432).

2. Definición del Servicio (docker-compose.yml)

La configuración del servicio de base de datos se define en el manifiesto de orquestación principal. Se utilizan volúmenes para garantizar la persistencia de los datos tras el reinicio de los contenedores.

```
# Servicio de Base de Datos
  app-db:
    image: postgres:15-alpine
    container_name: app-db
    restart: always
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: app_main_db
    volumes:
      - postgres_data:/var/lib/postgresql/data
      # Script de inicialización automática
      - ./init_db.sql:/docker-entrypoint-initdb.d/init.sql
    networks:
      - ai_net
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d app_main_db"]
      interval: 10s
      timeout: 5s
      retries: 5
```

3. Script de Inicialización (init.sql)

Para automatizar la creación de tablas al desplegar el proyecto por primera vez, se utiliza el siguiente script SQL estándar. Este archivo se monta en el directorio /docker-entrypoint-initdb.d/, lo que provoca que PostgreSQL lo ejecute automáticamente en el primer arranque.

-- Conexión a la base de datos: app_main_db

```
-- 1. Tabla de USUARIOS (Gestión de identidad y RBAC)
CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    hashed_password VARCHAR(255) NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    is_admin BOOLEAN DEFAULT FALSE, -- Rol: True=Admin, False=User
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. Tabla de SESIONES DE CHAT (Agrupación de hilos)
CREATE TABLE IF NOT EXISTS chat_sessions (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    title VARCHAR(255), -- Título autogenerado o manual
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Tabla de MENSAJES (Interacciones atómicas)
CREATE TABLE IF NOT EXISTS chat_messages (
    id SERIAL PRIMARY KEY,
    session_id INTEGER NOT NULL REFERENCES chat_sessions(id) ON DELETE CASCADE,
    role VARCHAR(20) CHECK (role IN ('user', 'assistant', 'system')),
    content TEXT NOT NULL,
    model_used VARCHAR(50), -- Auditoría del modelo (ej: Llama3)
    tokens_used INTEGER,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
-- 4. Índices de Optimización
-- Acelera la búsqueda de chats por usuario
CREATE INDEX IF NOT EXISTS ix_chat_sessions_user_id ON chat_sessions(user_id);
-- Acelera la carga del historial de mensajes
CREATE INDEX IF NOT EXISTS ix_chat_messages_session_id ON
chat_messages(session_id);
```

4. Verificación del Despliegue

Para validar que la estructura se ha creado correctamente en el entorno de producción, se pueden ejecutar los siguientes comandos de auditoría desde el host:

Acceder al contenedor:

```
docker exec -it app-db psql -U app_user -d app_main_db
```

Listar relaciones creadas:

```
\dt
```

```
app_main_db=# \dt
                List of relations
 Schema |        Name        | Type  |  Owner
--------+--------------------+-------+----------
 public | ai_models          | table | app_user
 public | chat_messages      | table | app_user
 public | chat_sessions      | table | app_user
 public | documents          | table | app_user
 public | message_feedback   | table | app_user
 public | users              | table | app_user
(6 rows)
```

```
app_main_db=# \d users
                                        Table "public.users"
      Column      |           Type           | Collation | Nullable |              Default
------------------+--------------------------+-----------+----------+-------------------------------------
 id               | integer                  |           | not null | nextval('users_id_seq'::regclass)
 email            | character varying        |           | not null |
 username         | character varying        |           |          |
 hashed_password  | character varying        |           | not null |
 is_active        | boolean                  |           |          |
 is_admin         | boolean                  |           |          |
 created_at       | timestamp with time zone |           |          | now()
Indexes:
    "users_pkey" PRIMARY KEY, btree (id)
    "ix_users_email" UNIQUE, btree (email)
    "ix_users_id" btree (id)
    "ix_users_username" UNIQUE, btree (username)
Referenced by:
    TABLE "chat_sessions" CONSTRAINT "chat_sessions_user_id_fkey" FOREIGN KEY (user_id) REFERENCES users(id)
    TABLE "documents" CONSTRAINT "documents_user_id_fkey" FOREIGN KEY (user_id) REFERENCES users(id)
```

```
app_main_db=# \d chat_messages
                                        Table "public.chat_messages"
   Column    |           Type           | Collation | Nullable |                Default
-------------+--------------------------+-----------+----------+-----------------------------------------
 id          | integer                  |           | not null | nextval('chat_messages_id_seq'::regclass)
 session_id  | integer                  |           |          |
 role        | character varying        |           | not null |
 content     | text                     |           | not null |
 tokens_used | integer                  |           |          |
 model_used  | character varying        |           |          |
 created_at  | timestamp with time zone |           |          | now()
Indexes:
    "chat_messages_pkey" PRIMARY KEY, btree (id)
    "ix_chat_messages_id" btree (id)
Foreign-key constraints:
    "chat_messages_session_id_fkey" FOREIGN KEY (session_id) REFERENCES chat_sessions(id)
Referenced by:
    TABLE "message_feedback" CONSTRAINT "message_feedback_message_id_fkey" FOREIGN KEY (message_id) REFERENCES chat_messages(id)
```

```
app_main_db=# \d chat_sessions
                                        Table "public.chat_sessions"
   Column   |           Type           | Collation | Nullable |                Default
------------+--------------------------+-----------+----------+-----------------------------------------
 id         | integer                  |           | not null | nextval('chat_sessions_id_seq'::regclass)
 user_id    | integer                  |           |          |
 title      | character varying        |           |          |
 created_at | timestamp with time zone |           |          | now()
 updated_at | timestamp with time zone |           |          |
Indexes:
    "chat_sessions_pkey" PRIMARY KEY, btree (id)
    "ix_chat_sessions_id" btree (id)
Foreign-key constraints:
    "chat_sessions_user_id_fkey" FOREIGN KEY (user_id) REFERENCES users(id)
Referenced by:
    TABLE "chat_messages" CONSTRAINT "chat_messages_session_id_fkey" FOREIGN KEY (session_id) REFERENCES chat_sessions(id)
```

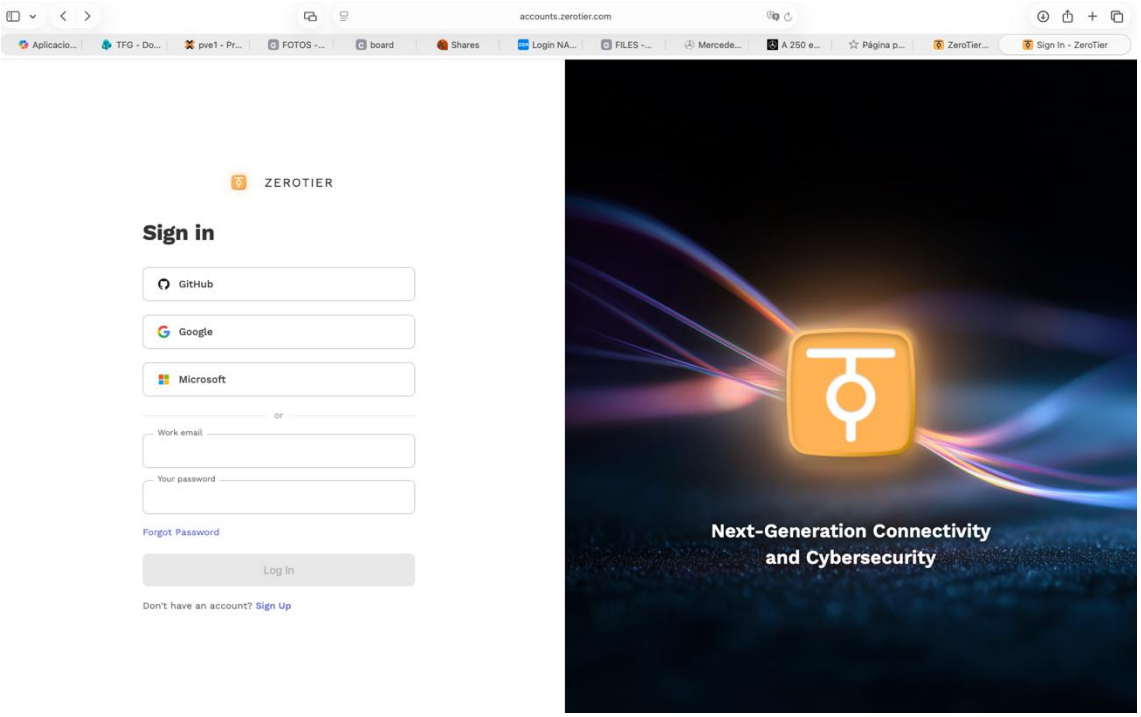
ANEXO II. Infraestructura y Redes

1. Configuración de la Red Privada Virtual (SD-WAN)

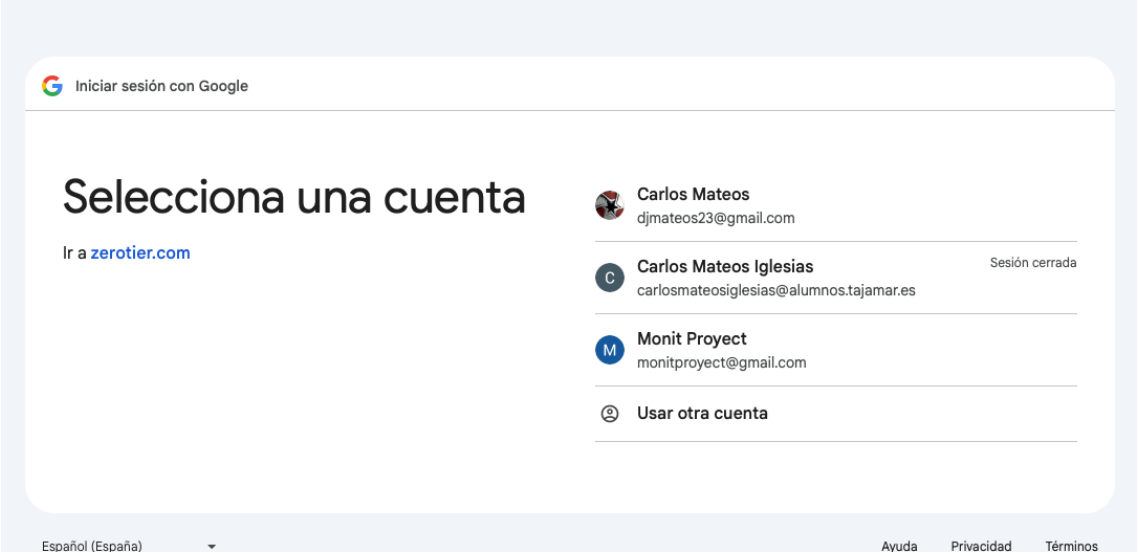
Vamos a realizar la instalación de ZeroTier, para poder acceder a la infraestructura desde cualquier parte. Para conseguir esto necesitamos un dominio registrado, una cuenta en zerotier, y una máquina en la nube con IP pública.

2. Vinculación a la Red (Join)

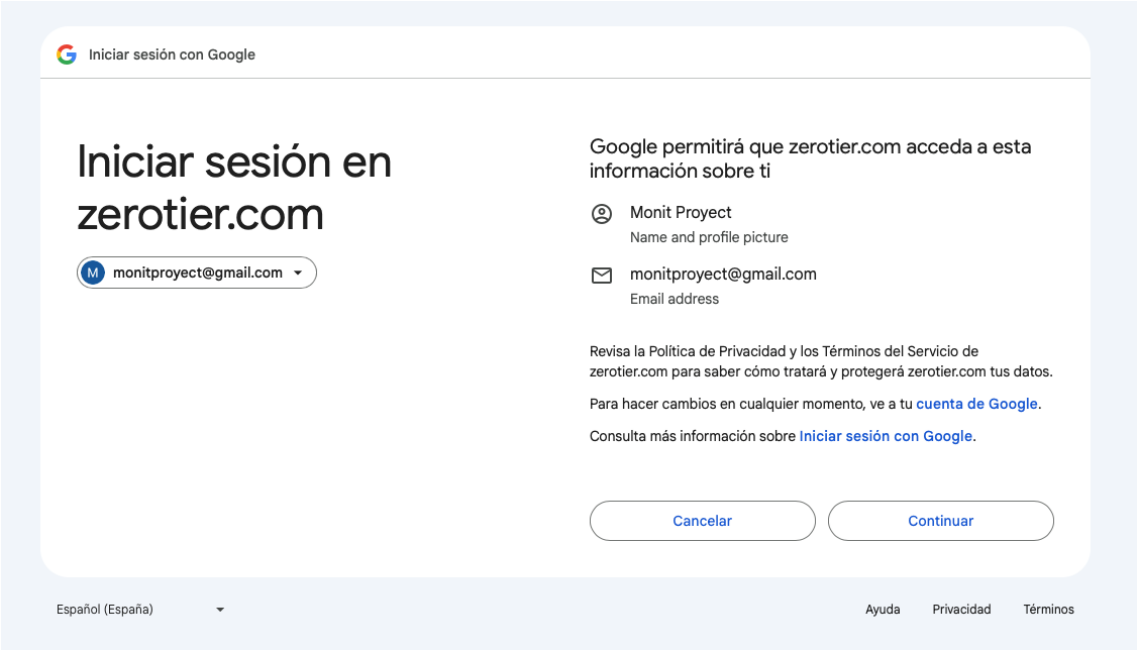
Accedemos a la página oficial de zerotier



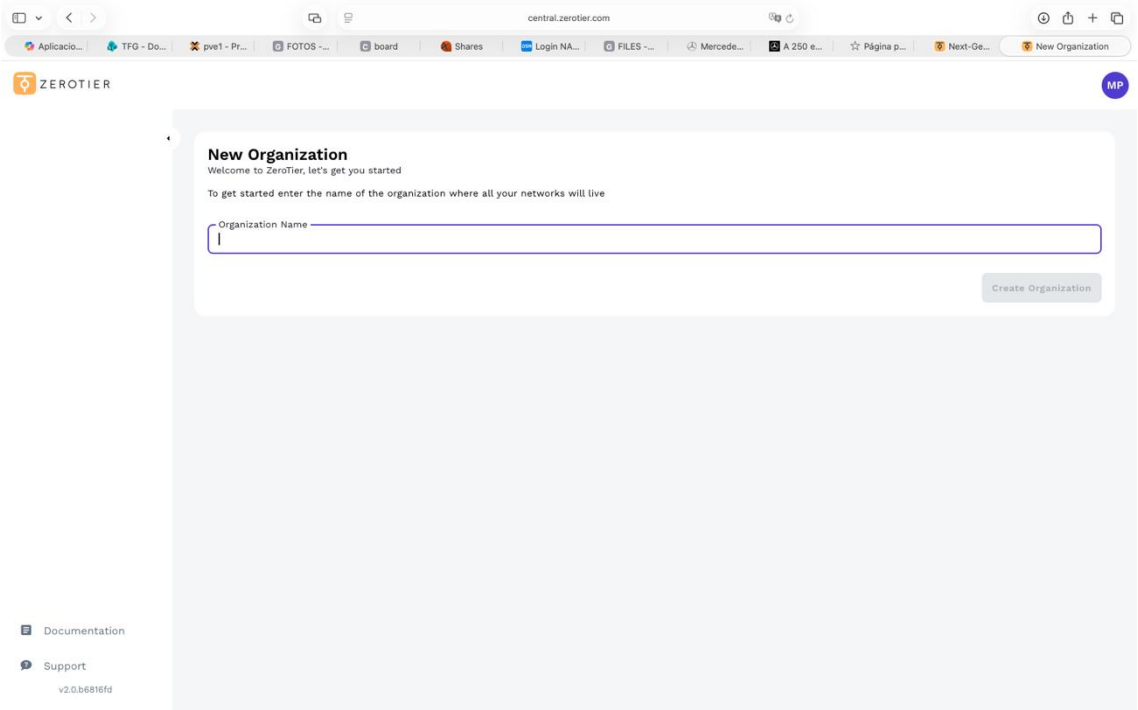
En mi caso me registraré con Google ya que es más cómodo para mi.



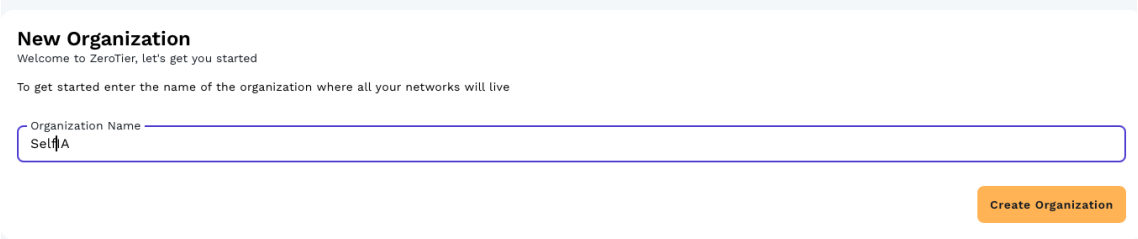
Inicio sesión con la cuenta Monit Proyect



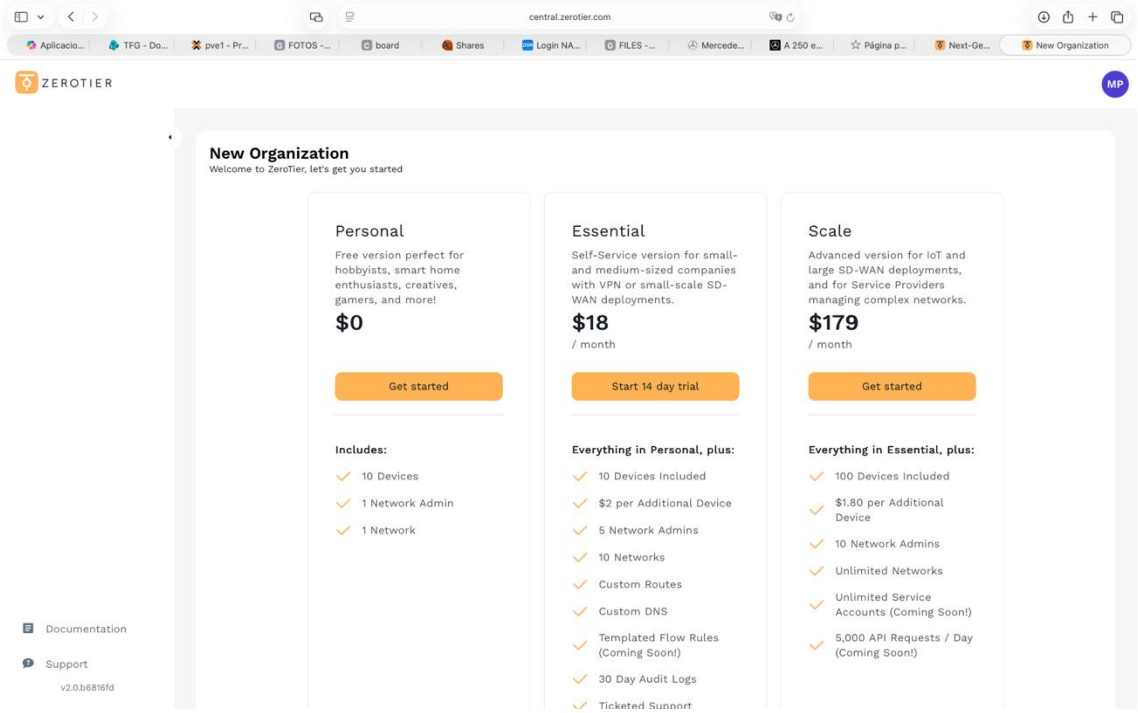
Le damos a continuar y ya tendríamos la cuenta de zerotier creada



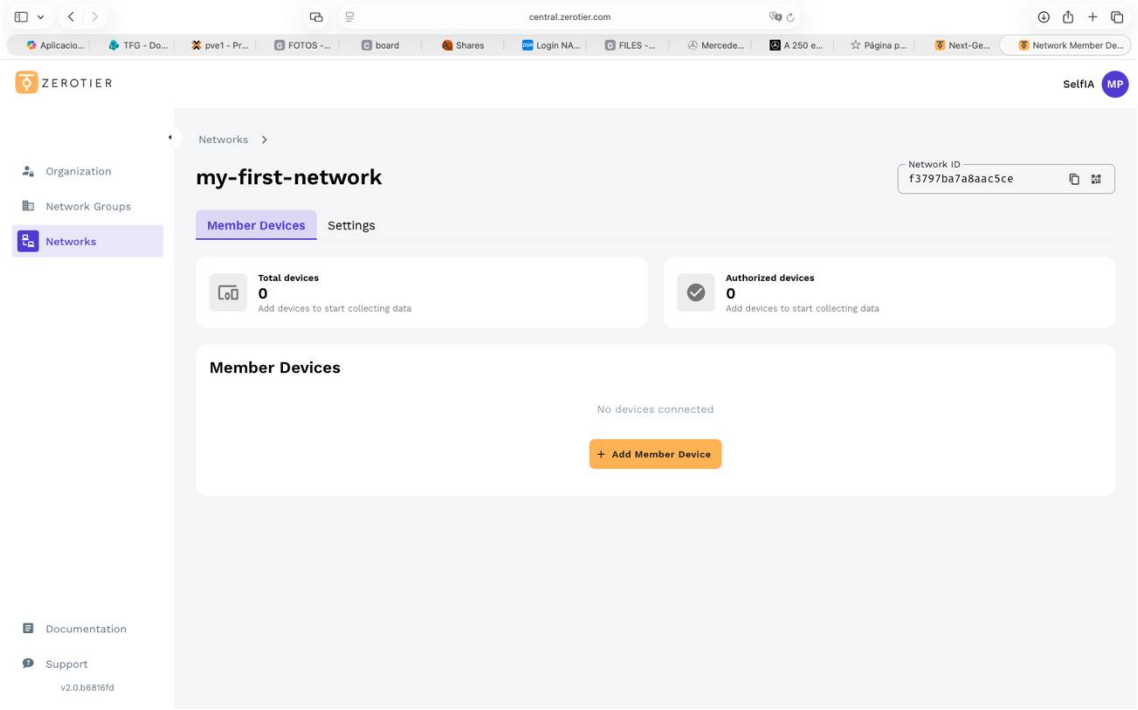
Una vez dentro le ponemos un nombre a la organización. Le



Una vez le asignemos nombre le daremos a crear organización. Con fines de pruebas y sin ser este el final del proyecto por ahora, usaremos una capa gratuita del SAS, es decir la personal.



Seleccionamos la personal, con idea del día de mañana expandir a una versión Enterprise.



Ahora nos debemos fijar en el Network ID, para poder añadir nuevos dispositivos.

3. Instalación del Cliente

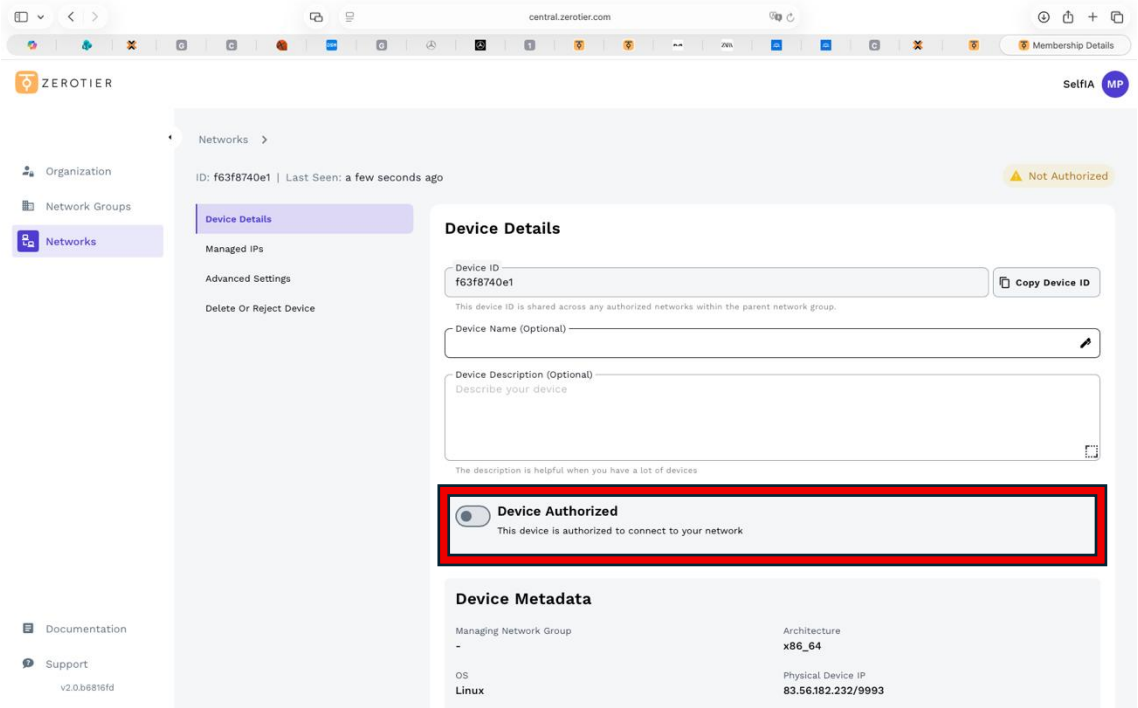
A continuación, nos vamos a nuestra máquina local y le instalamos el zerotier-cli

```
Last login: Wed Jan  7 11:06:43 2026 from 192.168.1.17
cmateos@debianMSI:~$ curl -s https://install.zerotier.com | sudo bash
[sudo] contraseña para cmateos:

*** ZeroTier Service Quick Install for Unix-like Systems

*** Tested OSes / distributions:

***   MacOS (10.13+) (just installs ZeroTier One.pkg)
***   Debian Linux (7+)
***   RedHat/CentOS Linux (6+)
***   Fedora Linux (16+)
***   SuSE Linux (12+)
***   Mint Linux (20+)
***   Kali Linux (2024.1+)

*** Supported architectures vary by OS / distribution. We try to support
*** every system architecture supported by the target.

*** Please report problems by opening a GitHub issue or Pull Request at:
*** https://github.com/zerotier/install.zerotier.com
*** Please include the content of `/etc/os-release` for your distribution.

*** Detecting Linux Distribution

*** Detected Debian Linux, creating /etc/apt/sources.list.d/zerotier.list
Des:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Des:2 http://security.debian.org/debian-security trixie-security InRelease [43,4 kB]
Des:3 http://deb.debian.org/debian trixie-updates InRelease [47,3 kB]
Des:4 http://security.debian.org/debian-security trixie-security/main Sources [117 kB]
Des:5 https://nvidia.github.io/libnvidia-container/stable/deb/amd64  InRelease [1.477 B]
Des:6 https://nvidia.github.io/libnvidia-container/experimental/deb/amd64  InRelease [1.489 B]
Des:7 http://deb.debian.org/debian trixie/main Sources [10,5 MB]
Des:8 https://download.docker.com/linux/debian trixie InRelease [32,5 kB]
Des:9 http://security.debian.org/debian-security trixie-security/main amd64 Packages [94,2 kB]
Des:10 http://security.debian.org/debian-security trixie-security/main Translation-en [59,3 kB]
Des:11 https://nvidia.github.io/libnvidia-container/stable/deb/amd64  Packages [23,3 kB]
Des:12 https://download.docker.com/linux/debian trixie/stable amd64 Packages [23,4 kB]
Des:13 http://deb.debian.org/debian trixie/non-free-firmware Sources [6.548 B]
Des:14 http://deb.debian.org/debian trixie/non-free Sources [75,9 kB]
Des:15 http://deb.debian.org/debian trixie/main amd64 Packages [9.670 kB]
Des:16 http://deb.debian.org/debian trixie/main Translation-en [6.484 kB]
Des:17 https://nvidia.github.io/libnvidia-container/experimental/deb/amd64  Packages [16,2 kB]
Des:18 http://deb.debian.org/debian trixie/non-free amd64 Packages [100 kB]
Des:19 http://deb.debian.org/debian trixie/non-free Translation-en [67,1 kB]
```

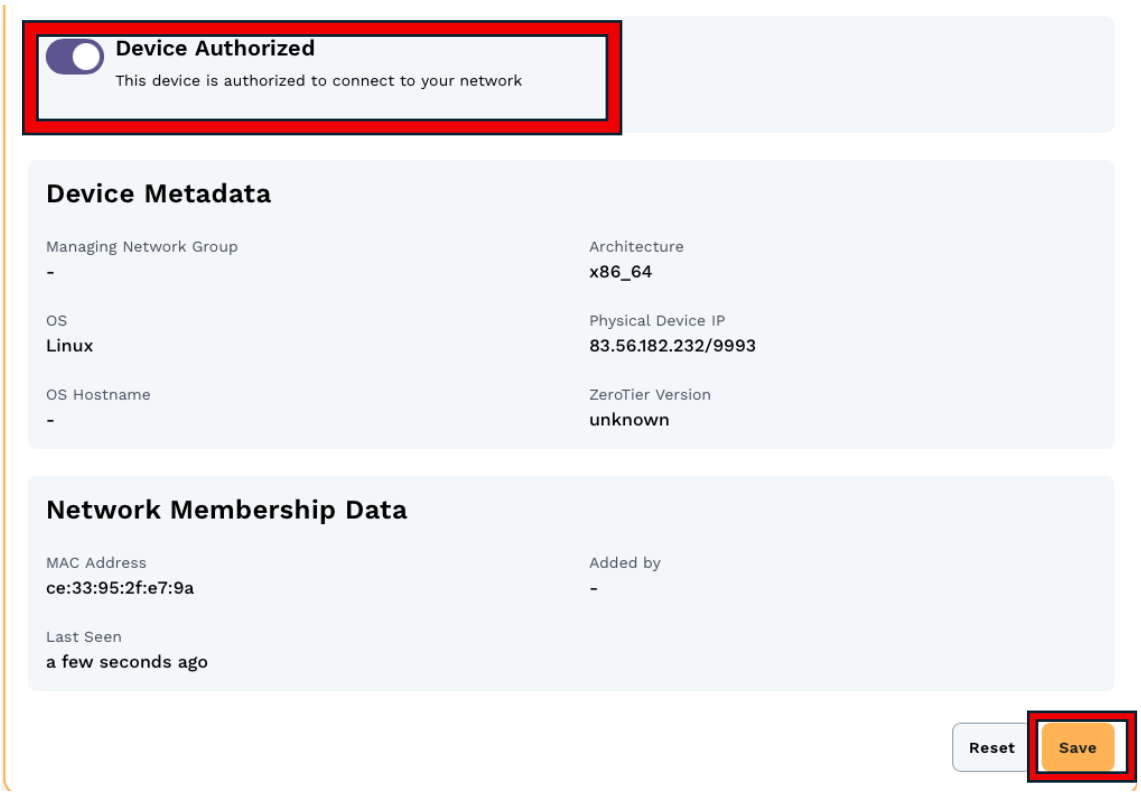
A continuación, hacemos sudo y como root ejecutamos el comando zeerotier-cli join (NetworkID)

```
cmateos@debianMSI:~$ zerotier-cli join f3797ba7a8aac5ce
-bash: zerotier-cli: orden no encontrada
cmateos@debianMSI:~$ sudo su -
root@debianMSI:~# zerotier-cli join f3797ba7a8aac5ce
200 join OK
root@debianMSI:~#
```

Una vez hecho esto, vamos a la página de zerotier para poder aceptar a este nuevo integrante



Le asignamos nombre y autorizamos el dispositivo

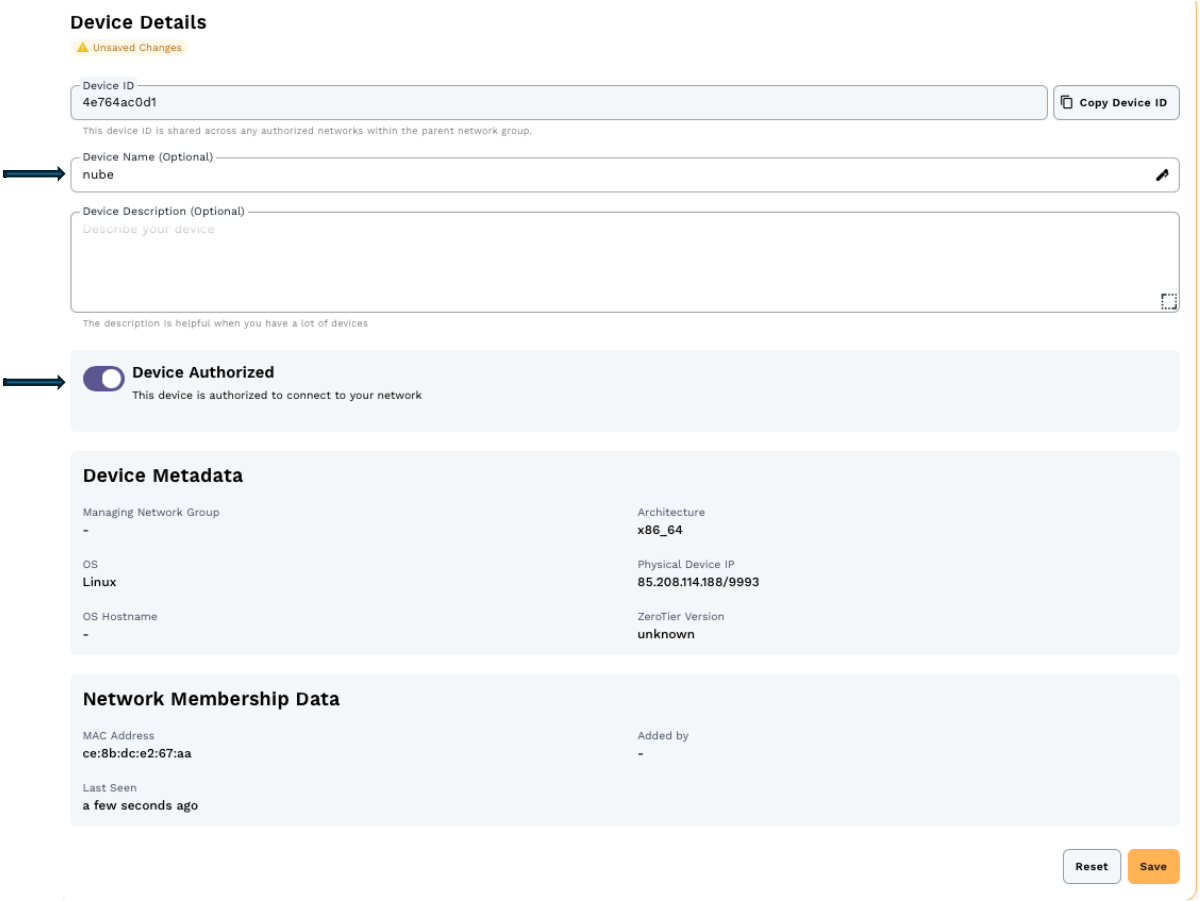


A continuación, nos vamos a una máquina que tenga IP pública estatica, en mi casa un lxc en un proveedor.

Lo instalamos como lo instalamos en la otra máquina, y lo unimos.

```
root@CarlitosGTI:~29 zerotier-cli status
200 info 4e764ac0d1 1.16.1 ONLINE
root@CarlitosGTI:~30 zerotier-cli join f3797ba7a8aac5ce
200 join OK
root@CarlitosGTI:~31
```

Lo aceptamos en la web.

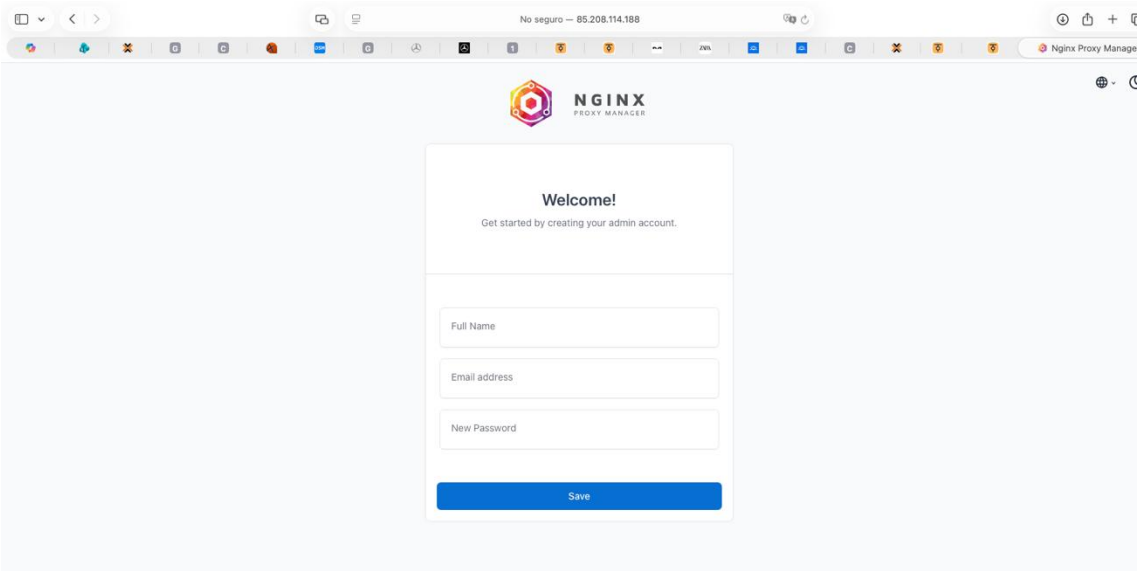


4. Orquestación de Contenedores (Docker)

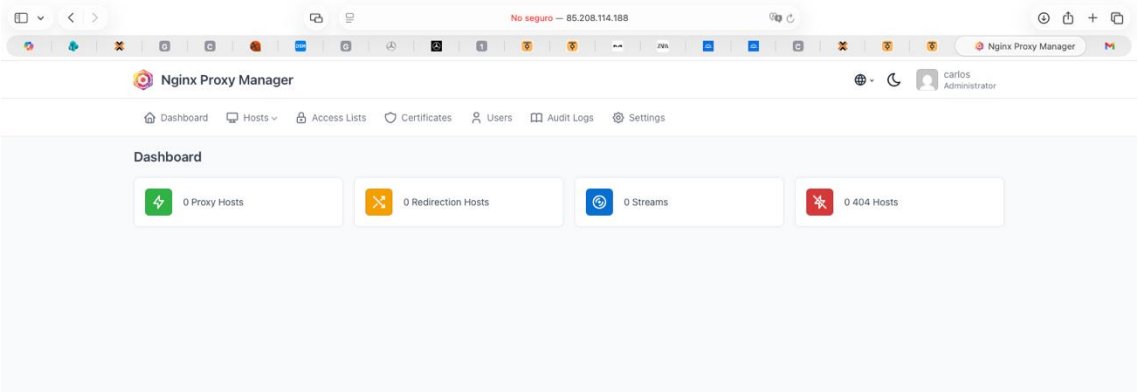
Una vez hecho esto, vamos a instalar Docker para posteriormente poder instalar un Ngnix Proxy Manager (de ahora en adelante NPM)

```
root@CarlitosGTI:~36 mkdir containers
root@CarlitosGTI:~37 nano ./containers/docker-compose.yml
root@CarlitosGTI:~38 cat !$
cat ./containers/docker-compose.yml
services:
  app:
    image: 'jc21/nginx-proxy-manager:latest'
    restart: unless-stopped
    environment:
      TZ: "Europe/Madrid"
    ports:
      - '80:80'
      - '81:81'
      - '443:443'
    volumes:
      - ./data:/data
      - ./letsencrypt:/etc/letsencrypt
root@CarlitosGTI:~39 cd containers/
root@CarlitosGTI:~/containers40 docker compose up -d
[+] up 33/35
 ⠼ Image jc21/nginx-proxy-manager:latest [⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿] 382.4MB / 382.4MB Pulling
```

Accedemos a la Ip pública de la máquina por el puerto 81 desde el navegador para iniciar la configuración de NPM

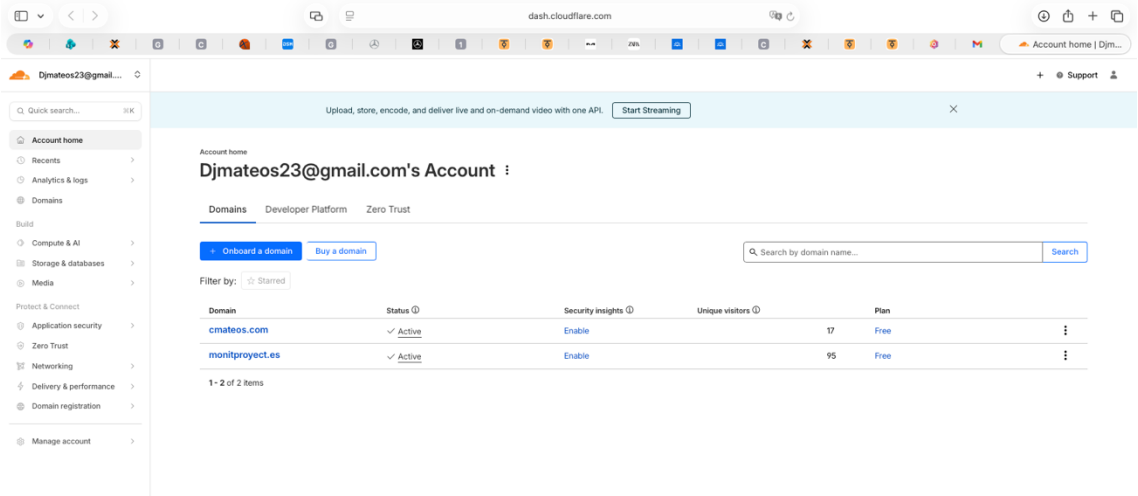


Una vez completada la info le damos en save, y tendríamos configurado el NPM

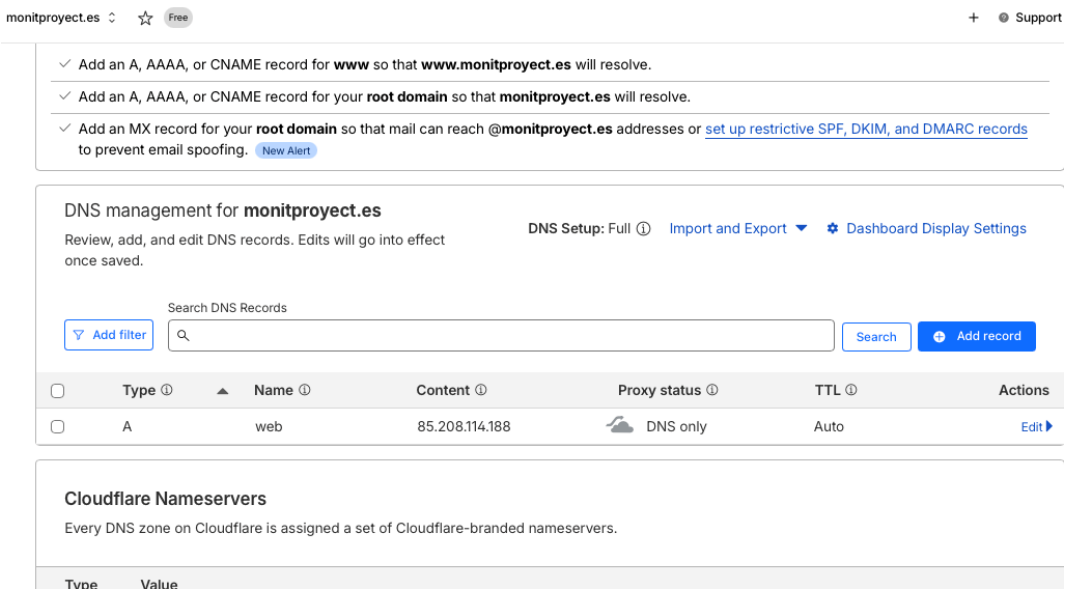


5. Configuración de túneles

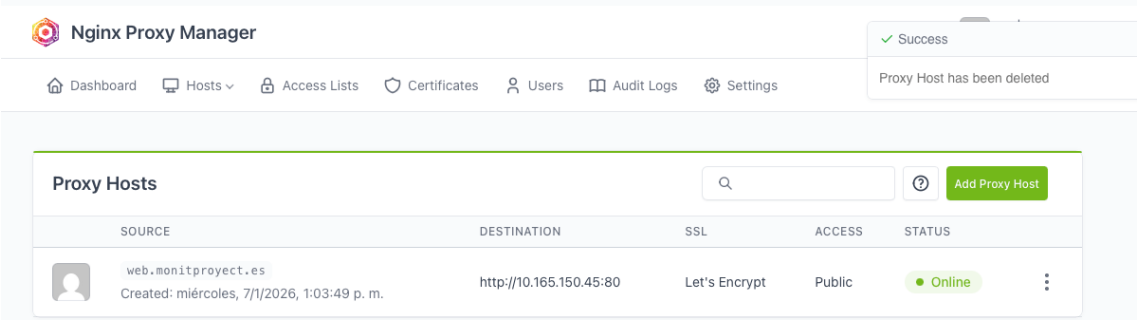
Ahora vamos a añadir los túneles para poder acceder por URL, para ello debemos acceder a clouflared



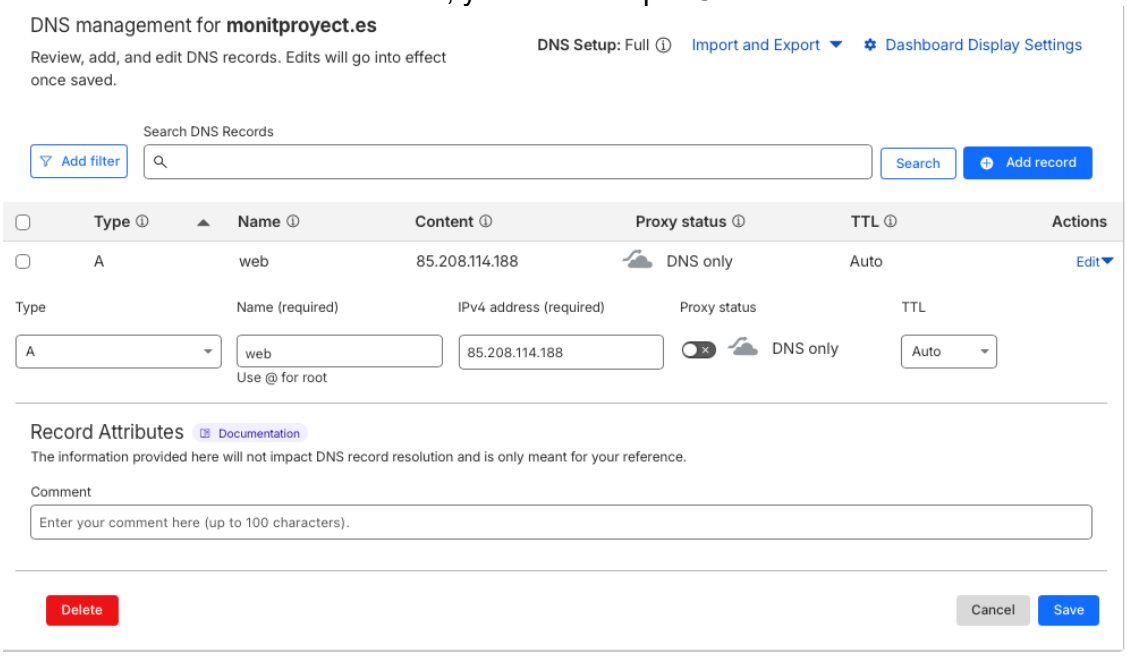
El primero que vamos a meter es la máquina nodo, para ello debemos poner la IP de la máquina cloud, ya que será la que hará de nexo de unión para los túneles de ZeroTier



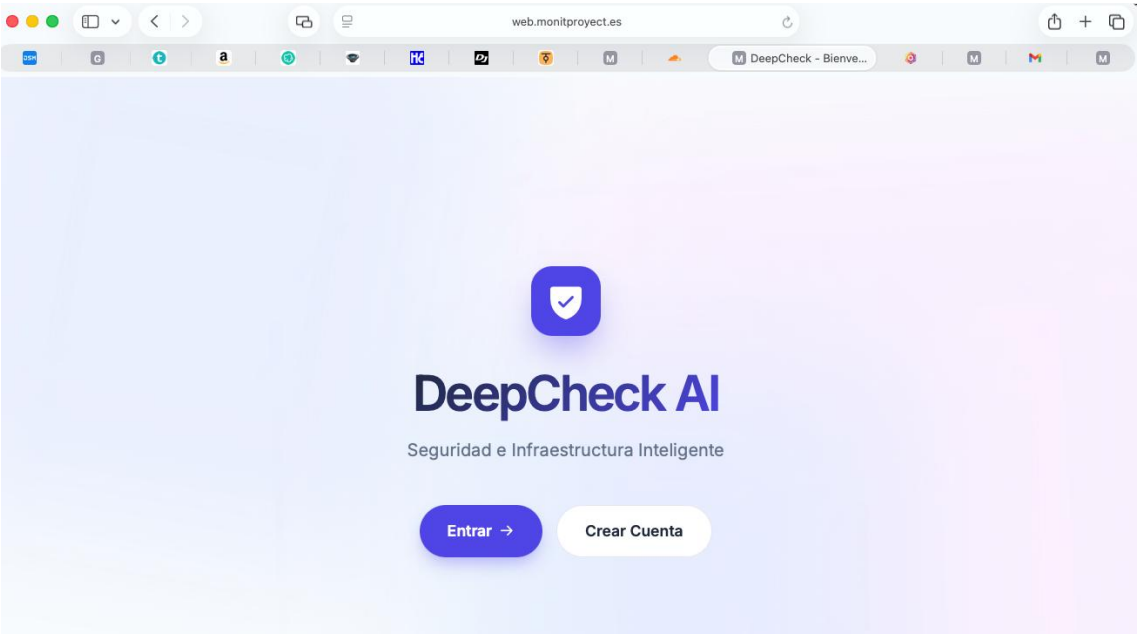
Una vez hecho, debemos ir al NPM y hacer la url y el puerto



Una vez creado en NPM lo creamos en cloudflare apuntando a la IP del servidor en la nube como hicimos anteriormente, y accedemos por URL



Probamos a acceder a la web:



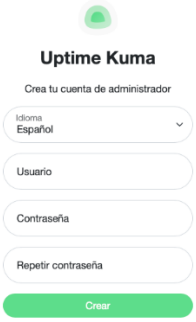
ANEXO III. Monitorización y Mantenimiento

1. Despliegue y Acceso Inicial

Configuración de uptimekuma, lo contenirezamos también por aquí el Docker-compose.yml

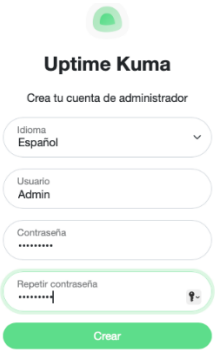
```
# Uptime Kuma  (disponibilidad)
  uptime-kuma:
    image: louislam/uptime-kuma:latest
    container_name: uptime-kuma
    networks: [ai_net]
    ports:
      - "3001:3001"
    volumes:
      - ./uptime-kuma:/app/data
    restart: unless-stopped
```

Accedemos al puerto 3001 en el que publica uptime su web. Y procedemos al registro

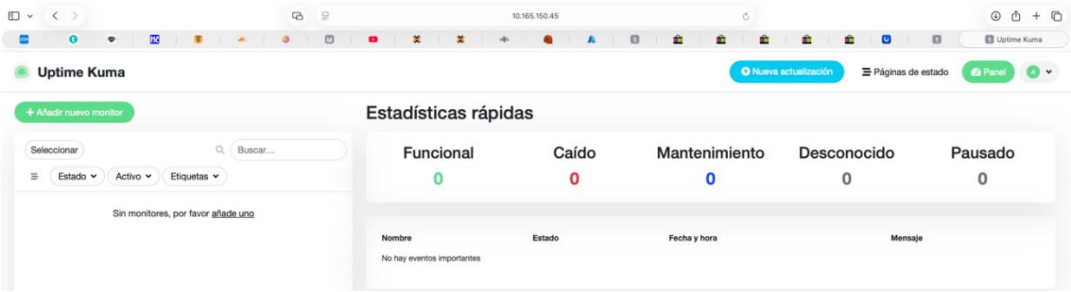


2. Configuración de Credenciales Administrativas

Creamos un usuario administrador con sus credenciales y le damos a crear

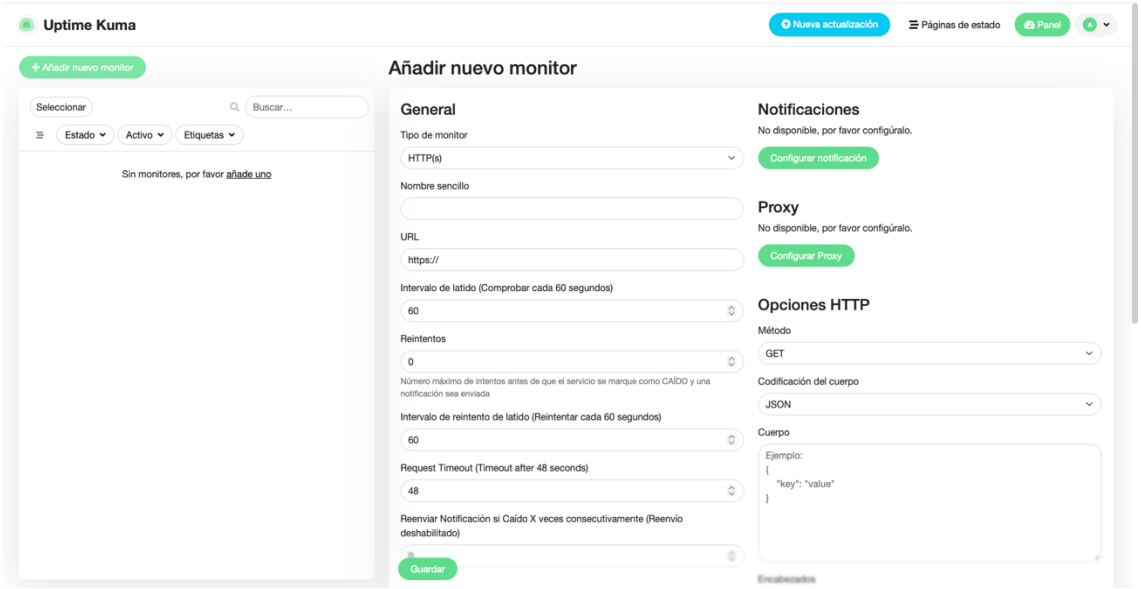


Una vez le demos a crear ya tendremos la web para personalizar y tirar uptimes



3. Definición de Sondas de Disponibilidad y Creación del panel de estado (Monitors)

Pulsaremos sobre Añadir nuevo monitor:



A continuación, los servicios que deseo monitorizar:

Monitor "API (docs)"

- Tipo: **HTTP**
- URL: http://fastapi:8000/docs
- Expected status: **200**
- Interval: **30s**
- Timeout: **8s**
- Retries: **2**

Ollama

- Tipo: **HTTP**
- URL: http://ollama:11434/api/tags
- Expected: **200**
- Interval: **60s**
- Timeout: **10s**

Open-WebUI

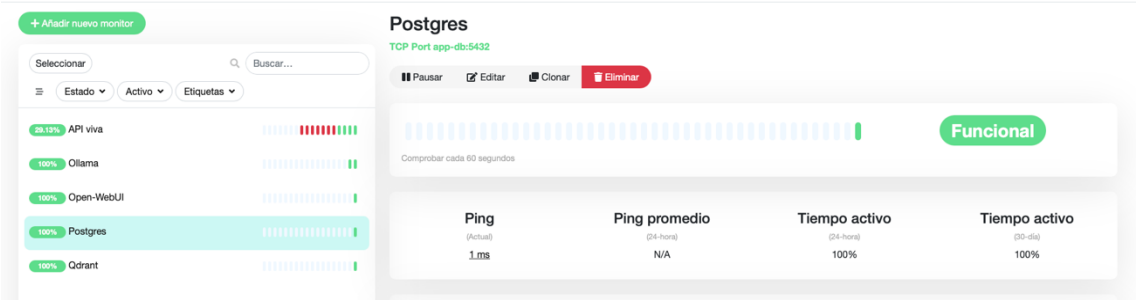
- Tipo: **HTTP**
- URL: http://open-webui:8080/
- Expected: **200**
- Interval: **60s**
- Timeout: **10s**

Qdrant

- Tipo: **HTTP**
- URL: http://qdrant:6333/
- Expected: **200**
- Interval: **60s**
- Timeout: **10s**

Monitor de Postgres

- Tipo: **TCP Port**
- Host: app-db
- Port: 5432
- Interval: 60s



¿Por qué Uptime Kuma está en la misma máquina y no en otra?

Se ha decidido **desplegar Uptime Kuma en el mismo servidor** por motivos técnicos y arquitectónicos:

1. **Monitorización real del estado interno**

   Si el sistema falla internamente (API, base de datos, Ollama), un monitor externo solo vería que “la web no responde”, pero no sabría por qué.

2. **Independencia del acceso externo**

   Si Cloudflare, el proxy cloud o Internet fallan, Uptime Kuma sigue funcionando y mostrando que el sistema interno está operativo.

3. **Arquitectura profesional por planos**

   Se distingue entre:

   - Plano interno (servicios reales)
   - Plano externo (experiencia del usuario)

Esto permite identificar con precisión dónde se origina una incidencia.

¿Por qué se accede a los servicios por red local Docker y no por URL?

Dentro del entorno Docker, los servicios se comunican mediante una **red interna (ai_net)** usando **nombres de contenedor**, no URLs públicas.

Ejemplos:

- http://fastapi:8000
- http://ollama:11434
- app-db:5432

Ventajas de este enfoque:

- **Mayor fiabilidad** (no depende de DNS externo)
- **Menor latencia**
- **Más seguridad** (servicios no expuestos)
- **Resolución automática por Docker DNS**
- **Independencia de IPs cambiantes**

Las URLs públicas solo se utilizan para:

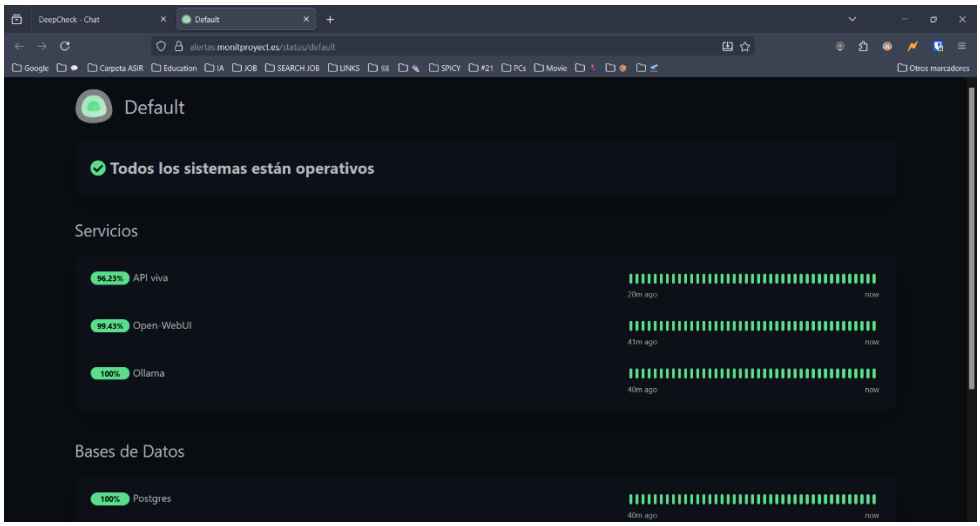
- Acceso de usuarios
- Monitorización externa (experiencia real desde Internet)

Esto sigue las mejores prácticas en arquitecturas de microservicios.

La arquitectura diseñada separa claramente:

- Inferencia de IA
- Lógica de aplicación
- Persistencia de datos
- Interfaz de usuario
- Monitorización

Cada componente cumple una función específica y se comunica mediante canales controlados, lo que resulta en una plataforma **robusta, escalable, segura y profesional**, alineada con escenarios reales de producción.



ANEXO IV. Cumplimiento Normativo (RGPD)

Cumplimiento del Reglamento General de Protección de Datos (RGPD)

Uno de los pilares fundamentales del proyecto DeepCheck IA es la privacidad y protección de los datos, aspecto crítico en cualquier sistema que maneje información sensible y que, además, integra inteligencia artificial generativa.

A diferencia de plataformas de IA de terceros, donde los datos introducidos por los usuarios pueden ser utilizados para entrenamiento de modelos o análisis externos, DeepCheck IA ha sido diseñado bajo el principio de IA privada y controlada, alineándose con los requisitos del Reglamento General de Protección de Datos (RGPD).

Principios RGPD aplicados en el proyecto

1-Minimización de datos

El sistema únicamente almacena los datos estrictamente necesarios para su funcionamiento:

- Correo electrónico.
- Contraseña (almacenada de forma cifrada mediante hashing).
- Rol de usuario (administrador o usuario estándar).

No se almacenan:

- Datos personales innecesarios.
- Conversaciones con la IA de forma permanente (salvo ampliación futura).
- Información sensible no requerida por el servicio.

Esto cumple el principio de minimización de datos recogido en el RGPD.

2 - Limitación de la finalidad

Los datos recogidos se utilizan exclusivamente para:

- Autenticación de usuarios.
- Control de acceso al sistema.
- Gestión de permisos (RBAC).

No se reutilizan los datos para:

- Entrenamiento de modelos.
- Análisis comerciales.
- Cesión a terceros.

3 -Integridad y confidencialidad

Se han aplicado múltiples capas de seguridad:

- Hashing de contraseñas mediante algoritmos seguros (bcrypt).
- Autenticación mediante tokens JWT, evitando credenciales persistentes en el cliente.

- Infraestructura aislada, donde la base de datos y el motor de IA no son accesibles desde Internet.
- Cifrado en tránsito mediante HTTPS (TLS), gestionado por el proxy inverso.

Esto garantiza la confidencialidad y evita accesos no autorizados.

4 - No cesión de datos a terceros

Uno de los aspectos más relevantes del proyecto es que:

- Los modelos de IA se ejecutan en local.
- No existe comunicación con APIs externas de IA.
- No se envían prompts ni datos a proveedores externos.

Esto elimina uno de los mayores riesgos de incumplimiento del RGPD en soluciones de IA comerciales.

5 - Control de acceso y trazabilidad

El sistema implementa:

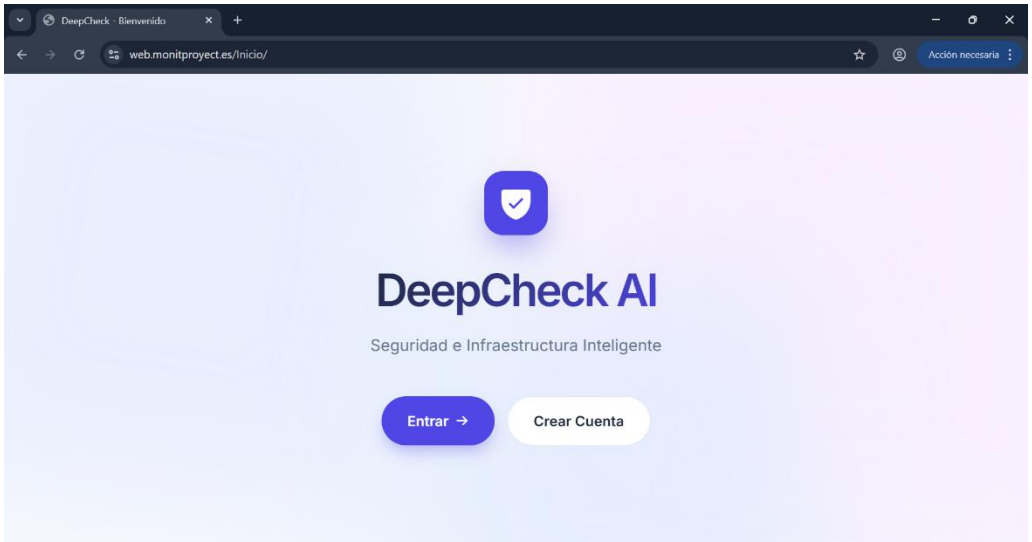
- Control de Acceso Basado en Roles (RBAC).
- Separación clara entre usuarios estándar y administradores.
- Endpoints protegidos según permisos.

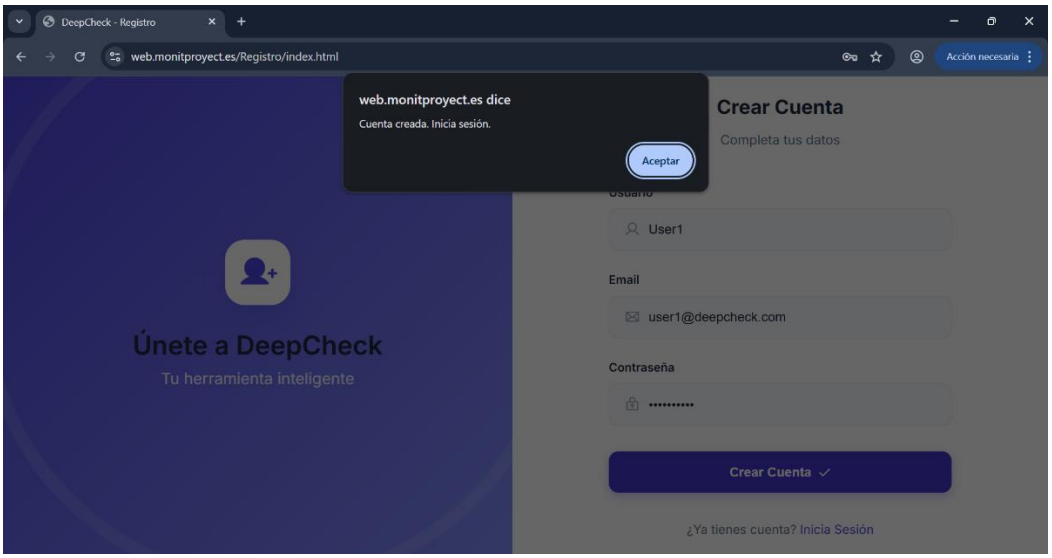
Esto asegura que cada usuario solo pueda acceder a la información y funcionalidades que le corresponden.

ANEXO V. Guía Rápida de Usuario

1. Acceso a la Plataforma

El usuario debe acceder a la URL proporcionada por el administrador. En la pantalla de bienvenida, seleccionará "Entrar" si ya tiene credenciales o "Crear Cuenta" para nuevos registros.

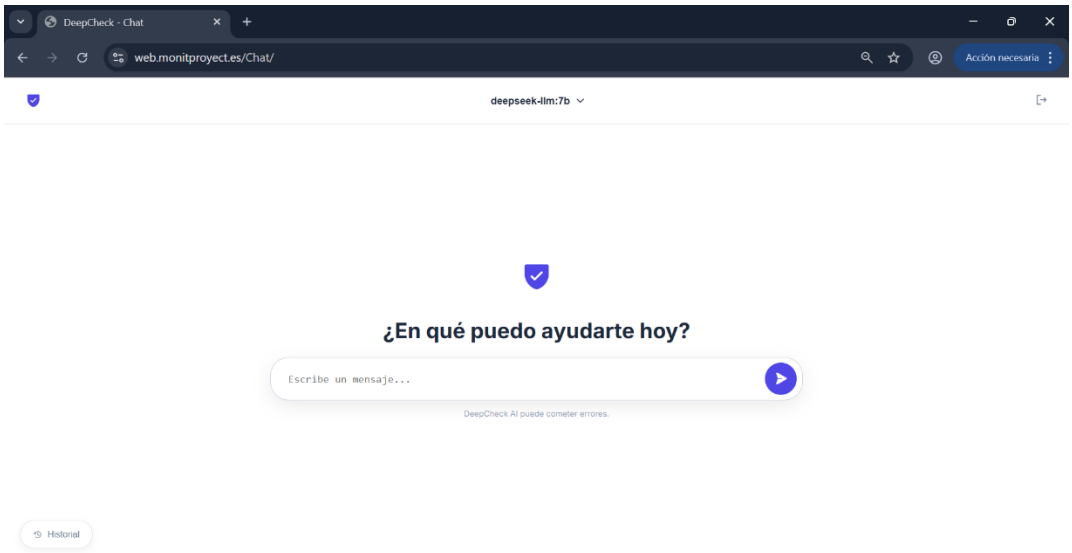


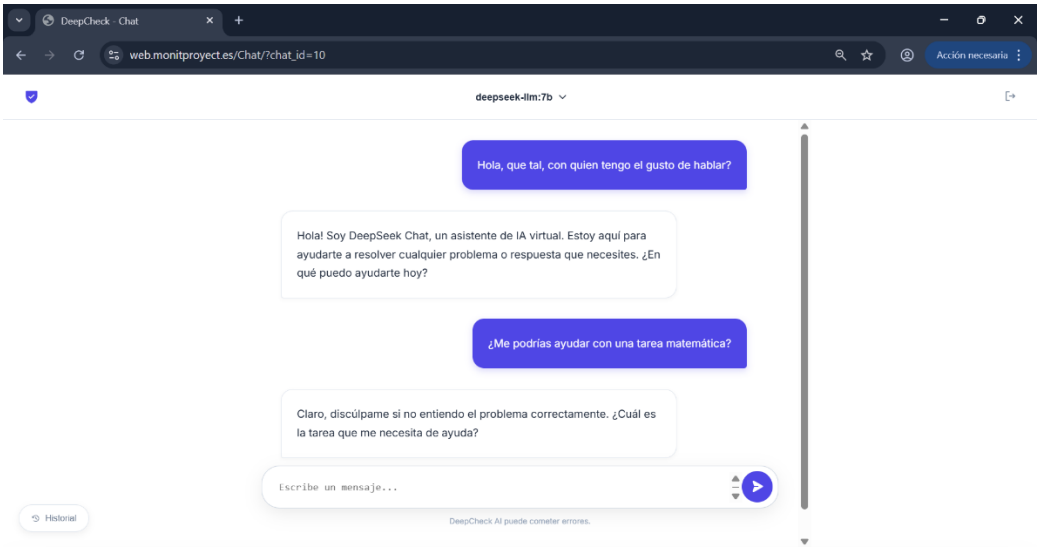


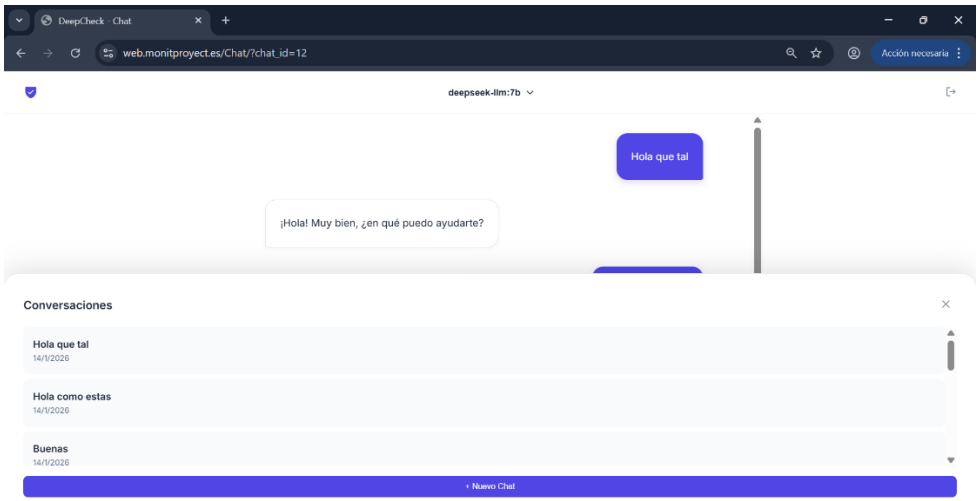
2. Interfaz de Chat

Una vez autenticado, el usuario accede al panel principal.

- Barra Lateral**:** Permite navegar entre conversaciones antiguas o iniciar un "Nuevo Chat".
- Área de Texto**:** Situada en la parte inferior, permite introducir el prompt.
- Selección de Modelo**:** En la parte superior, un desplegable permite cambiar entre Llama3 (rápido) o Mistral (preciso) según la necesidad.

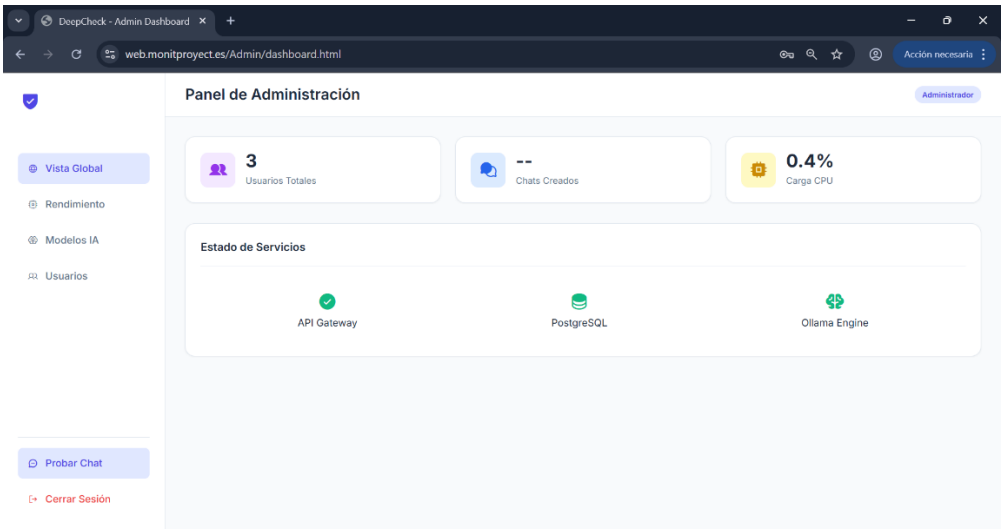

DeepCheck - Chat
web.monitproyect.es/Chat/
Acción necesaria
deepseek-llm:7b
¿En qué puedo ayudarte hoy?
Escribe un mensaje...
DeepCheck AI puede cometer errores.
Historial

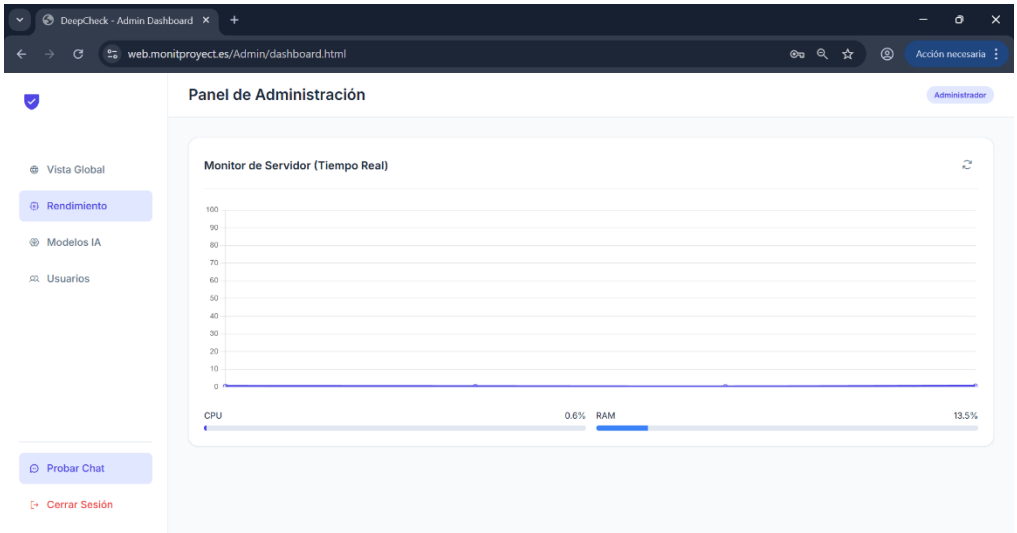

DeepCheck - Chat
web.monitproyect.es/Chat/?chat_id=10
Acción necesaria
deepseek-llm:7b
Hola, que tal, con quien tengo el gusto de hablar?
Hola! Soy DeepSeek Chat, un asistente de IA virtual. Estoy aquí para ayudarte a resolver cualquier problema o respuesta que necesites. ¿En qué puedo ayudarte hoy?
¿Me podrías ayudar con una tarea matemática?
Claro, discúlpame si no entiendo el problema correctamente. ¿Cuál es la tarea que me necesita de ayuda?
Escribe un mensaje...
Historial
DeepCheck AI puede cometer errores.

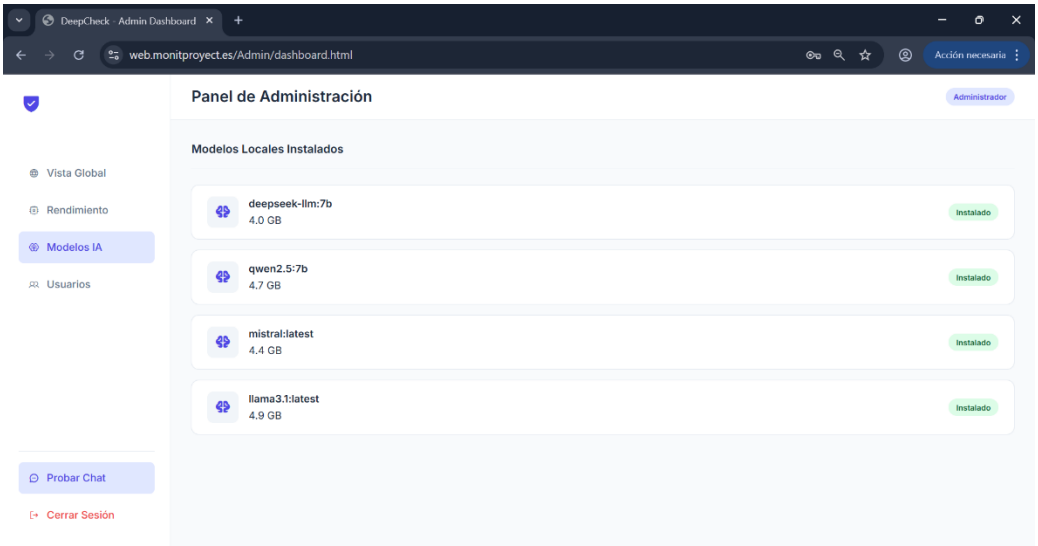

DeepCheck - Chat
web.monitproyect.es/Chat/?chat_id=12
Acción necesaria
deepseek-llm:7b
Hola que tal
¡Hola! Muy bien, ¿en qué puedo ayudarte?
Conversaciones
Hola que tal
14/1/2026
Hola como estas
14/1/2026
Buenas
14/1/2026
+ Nuevo Chat

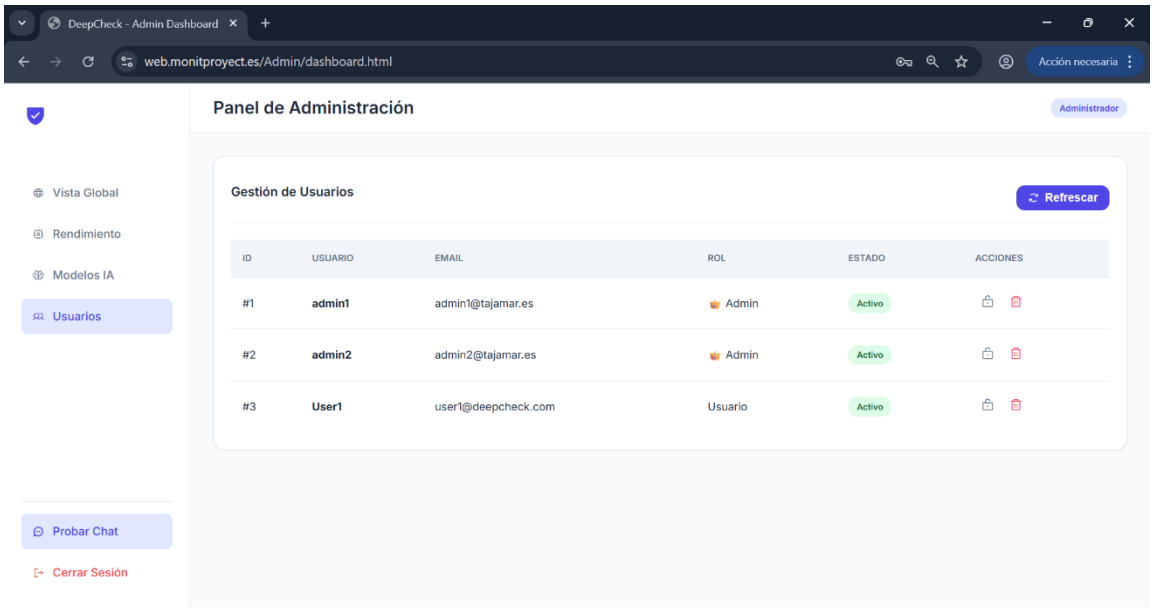
3. Panel de Administración (Solo Admins)

Los usuarios con rol de administrador verán un botón extra en la esquina superior derecha ("Dashboard"). Al pulsarlo, accederán a las gráficas de consumo de CPU/RAM y a la tabla de gestión de usuarios para bloquear o eliminar cuentas sospechosas.








DeepCheck - Admin Dashboard
web.monitproyect.es/Admin/dashboard.html
Acción necesaria
Panel de Administración
Administrador
Vista Global
Rendimiento
Modelos IA
Usuarios
Probar Chat
Cerrar Sesión
Gestión de Usuarios
Refrescar
ID
USUARIO
EMAIL
ROL
ESTADO
ACCIONES
#1
admin1
admin1@tajamar.es
Admin
Activo
#2
admin2
admin2@tajamar.es
Admin
Activo
#3
User1
user1@deepcheck.com
Usuario
Activo